

分类号： TP391

密 级： 公开

单位代码： 10431

学 号： 10431210661

齐鲁工业大学

硕士学位论文

(专业学位)

基于 API 行为特征的恶意代码检测技术研究

作 者 姓 名 张雨馨

领 域 计算机技术

所 在 学 部 计算机科学与技术学部

指导教师姓名 杨淑棉

专业技术职务 副研究员

2024 年 06 月 06 日

**A Thesis Submitted for the Application of
the Master's Degree of Engineering**

**Research on Malicious Code Detection
Technology Based on API Behavior
Characteristics**

Candidate: Yuxin Zhang

Specialty: Computer Technology

Supervisor: Associate Professor Shumian Yang

Qilu University of Technology, Jinan, China

June, 2024

目 录

第 1 章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 研究现状.....	2
1.3 主要研究内容.....	5
1.4 论文组织结构.....	6
第 2 章 恶意代码检测相关基础理论	8
2.1 恶意代码相关理论.....	8
2.1.1 恶意代码介绍.....	8
2.1.2 恶意代码反检测技术.....	8
2.2 恶意代码检测相关技术.....	10
2.2.1 静态分析技术.....	10
2.2.2 动态分析技术.....	10
2.2.3 混合分析技术.....	13
2.3 机器学习与深度学习算法	14
2.3.1 机器学习算法.....	14
2.3.2 深度学习算法.....	15
2.4 本章小结.....	22
第 3 章 基于 API 调用序列的恶意代码检测方法.....	23
3.1 引言.....	23
3.2 方法框架.....	23
3.3 数据预处理.....	24
3.4 序列特征嵌入.....	25
3.5 名称特征嵌入.....	26
3.6 CNNs-BiGRU 模型	27
3.7 实验与分析.....	29
3.7.1 实验环境与数据.....	29
3.7.2 评价方法与指标.....	31
3.7.3 实验结果分析.....	32
3.8 本章小结.....	37
第 4 章 基于 API 异构参数增强的恶意代码检测方法.....	38
4.1 引言.....	38
4.2 方法框架.....	39
4.3 数据预处理.....	39
4.4 API 序列模型	41

4.4.1 API 序列嵌入.....	41
4.4.2 双向时域卷积网络.....	42
4.5 API 参数模型	43
4.5.1 API 参数嵌入.....	43
4.5.2 轻量化神经网络模型.....	44
4.6 特征融合.....	45
4.7 实验与分析.....	46
4.7.1 实验环境与数据.....	46
4.7.2 评价方法与指标.....	47
4.7.3 实验结果分析.....	47
4.8 本章小结.....	54
第 5 章 总结与展望	55
5.1 总结.....	55
5.2 展望.....	56
参考文献.....	57

摘 要

在互联网高速发展的背景下，近年来恶意代码数量快速增长，其传播方式不断更新，已然成为网络空间中最主要的攻击方式之一。因此，对恶意代码进行高效且精准的检测，对于保障网络安全至关重要。恶意代码检测技术主要分为静态分析技术和动态分析技术。静态分析技术能够快速扫描大量文件以识别恶意代码，但却容易受到加壳、混淆等方式的干扰。动态分析技术通过实际运行代码观察其执行行为，尤其是应用程序接口（API）调用序列，解决了静态分析难以发现隐藏逻辑和恶意行为的局限性。然而，现有恶意代码动态检测方法往往存在对 API 调用序列信息分析不充分、对于 API 的运行参数信息缺乏深入研究以及训练模型计算资源需求高的问题。

本文针对上述问题展开研究工作，主要贡献体现在以下两个方面：

（1）针对现有方法对 API 调用序列信息分析不充分的问题，本文提出了一种基于 API 调用序列的恶意代码检测方法。该方法首先利用词嵌入技术对 API 调用序列进行向量化表示，以学习调用序列的上下文语义信息。然后基于 API 调用名称的结构特性对其重新描述，并通过词嵌入技术进行表示，同时结合调用名称的统计特征，以丰富向量表示的信息。最后通过融合多尺度卷积与双向门控循环单元，设计 CNNs-BiGRU 模型，旨在全面学习 API 序列的局部和全局特征。本文在公开的大型数据集上对该方法进行评估，实验结果表明，该方法能够充分提取 API 调用信息，从而有效提高检测模型的准确率。

（2）针对现有方法未能深入研究不同类型 API 运行参数的语义信息以及模型计算复杂度高的问题，本文提出了一种基于 API 异构参数增强的恶意代码检测方法。该方法从 API 序列与 API 参数两个方面展开分析，首先，运用段落向量化技术将不同长度的 API 序列转化为统一维度的矢量形式，并设计了双向时域卷积网络，以较低的计算复杂度捕捉序列前后向长距离依赖关系。其次，根据 API 异构类型参数的功能结构对其进行分解，并利用轻量化模型捕捉参数间语义的内在联系。最后，融合序列特征和参数特征，全面地理解程序行为。实验结果表明，该方法获得了较高的检测准确率，有效提高了检测模型的性能。

关键词：恶意代码检测；动态分析；深度学习；API 调用序列

ABSTRACT

With the rapid development of the Internet, the number of malicious codes has been growing rapidly in recent years. Their propagation methods are constantly updated, which has become one of the most important attacks in cyberspace. Therefore, it is crucial to ensure network security by efficiently and accurately detecting malicious codes. Malicious code detection technology is mainly divided into static analysis technology and dynamic analysis technology. Static analysis technology can quickly scan a large number of files to identify malicious code, but it is easily interfered with by packing and obfuscation. Dynamic analysis techniques observe the execution behavior of the code by actually running it, especially the application programming interface (API) call sequences, which solves the limitation of static analysis that makes it difficult to discover hidden logic and malicious behaviors. However, the existing methods for dynamic detection of malicious code usually suffer from insufficient analysis of API call sequence information, lack of in-depth research on the runtime parameter information of APIs, and high computational resource requirements for training models.

This thesis develops a research work to address the above issues and its contributions are mainly in the following two aspects:

(1) To address the problem of inadequate analysis of API call sequence information in existing methods, this thesis proposes a malicious code detection method based on API call sequences. The method first utilizes the word embedding technique to vectorize the representation of API call sequences in order to learn the contextual semantic information of the call sequences. Then the API call names are re-described based on their structural properties and represented by word embedding techniques while combining the statistical features of the call names to enrich the information of the vector representation. Finally, the CNNs-BiGRU model is designed by fusing multi-scale convolution with bi-directional gated recurrent units, aiming to comprehensively learn the local and global features of API sequences. In this thesis, the method is evaluated on a publicly available large-scale dataset, and the experimental results show that the method can fully extract the API call information, which effectively improves the accuracy of the detection model.

(2) To address the issues of existing methods failing to deeply explore the semantic information of different types of API operational parameters and the high computational complexity of models, this thesis proposes a malicious code detection method based on the enhancement of API heterogeneous parameters. The method is analyzed from two aspects: API sequences and API parameters. First, the paragraph vectorization technique is used to transform API sequences of different lengths into vector forms of uniform dimensions, and a bidirectional Temporal Convolutional Network is designed to capture the sequence backward and forward long-distance dependencies with low computational complexity. Second, decompose the parameters of API heterogeneous types according to their functional structures, and capture the semantic intrinsic connections among parameters using a lightweight model. Finally, the sequence features and parameter features are fused to provide a comprehensive understanding of program behavior. Experimental results show that the method obtains high detection accuracy and effectively improves the performance of the detection model.

Keywords: Malicious Code Detection, Dynamic Analysis, Deep Learning, API Call Sequences

第 1 章 绪论

1.1 研究背景及意义

随着网络信息技术的飞速发展和全面普及，互联网极大地改善了人们的生活质量。然而，互联网的开放性也催生了一系列不容忽视的安全隐患。计算机技术的繁荣推动了恶意代码的发展，其威胁力和复杂性日益增强，常常超越了传统防御系统的应对能力。尽管信息技术和网络安全防御取得了一些进步，但恶意代码攻击层出不穷，给网络空间的安全与稳定带来了极大的挑战。特别是在当今数字经济、人工智能、云计算、大数据等新技术和新应用正快速发展的时代，网络安全作为发展的基石，对保障经济社会稳定运行和维护广大人民群众的利益至关重要。

恶意代码行业不断发展，其规模逐步壮大，恶意代码的数量呈现出明显的上升趋势。根据国家互联网应急中心（CNCERT）公布的数据显示，2021 年内，我国国内遭受恶意代码侵害的计算机数量达到了约 446 万台，相较于前一年同比增长 46.8%。此外，SonicWall 在 2023 年发布的网络威胁报告^[1]也进一步证实了这一趋势，报告指出，在经历了自 2018 年至 2020 年连续三年的下降周期后，恶意代码的总量在 2022 年出现了逆转。2022 年，物联网（IoT）恶意代码数量增长了 87%，恶意代码数量在 2023 年 6 月更是达到 5.75 亿次的峰值，比 1 月份的 3.95 亿次增加了 46%。考虑到执行环境，Windows 操作系统（Operating System, OS）仍是迄今为止恶意代码针对性最强的操作系统，是恶意代码广泛针对的目标^[2]。每年有超过 1.14 亿个新的恶意代码被开发出来，其中超过 78% 应用于 Windows 系统。恶意代码样本的快速传播给反恶意代码系统构成了严峻考验。攻击者通过侵入主机系统，对操作系统乃至整个网络构成影响与损害，造成敏感数据泄露等严重后果。

科技的飞速发展使得恶意代码表现出了更强大和复杂的特性^[3,4]。网络犯罪分子通过一系列先进的恶意代码对抗技术，如加壳、死代码插入、代码转置、字符串加密和解密、反调试等手段，不断增强其隐蔽性和攻击性，从而成功逃避传统检测方法的追踪。恶意代码的威胁涉及领域广泛，不仅可能会给用户的信息安全带来危害，更可能对国家的能源安全 and 经济稳定造成不可估量的巨大损失。因此，如何实现对恶意代码准确和高效的检测，对于维护国家的网络安全和推进网络强国的建设至关重要。这不仅在理论层面上具有极其重要的研究意义，同时在实际应用中也展现出了很高的价值。

1.2 研究现状

随着计算机技术的不断发展和广泛应用，恶意代码的威胁日益严峻，系统受到恶意代码感染和遭受攻击的情况正在逐渐加剧。因此，迫切需要采取有效措施来检测和防范恶意代码，以维护网络安全。目前主流的检测方法包括静态分析检测方法、动态分析检测方法、混合分析检测方法和基于自然语言处理的检测方法，本节将对这四种检测方法在国内外恶意代码检测领域的研究现状展开叙述。

（1）静态分析检测方法

静态分析方法作为一种重要的恶意代码检测技术手段，其独特之处在于并不需要实际运行或执行样本代码。这种方法通过直接解析和分析程序的源代码或二进制文件结构，从而识别和提取恶意代码的特征。因此，静态分析方法在检测过程中展现出了安全性高、检测速度快以及检测开销低等优点。

Moskovitch 等人^[5,6]提出了基于操作码序列的恶意代码分类方法，使用不同维度的 N-gram 操作码序列构建特征集。使用文档频率、信息增益和 Fisher score 等不同方法，确定基于 2-grams 的操作码序列能够获得更好的分类性能。该方法解决了数据不均衡时准确率低的问题。Shabtai 等人^[7]提出了一种基于恶意代码静态特征的检测方法，通过机器学习模型对其进行分类，在保持较低的假阳性率的同时取得了较高的准确率。该方法运用 N-grams 算法，提取恶意代码的指令层和字节级特征，并借助多个分类器进行训练。对这些分类器的分类结果使用一种主动学习机制以及加权算法，以保持较高的检测准确率。然而，该方法是结合了几种简单的机器学习分类模型的结果，检测精度需要提高。Sami 等人^[8]提出了一种基于数据挖掘的恶意代码检测方法。该方法分析并提取可移植可执行的文件（PE 文件）的静态应用程序接口（API）调用，通过 Fisher score 评分选出最具区别性的 API 调用，来提高分类精度。最后使用随机森林进行分类，准确率达到 98.3%。该方法虽然在不平衡的数据集中取得不错的表现，但对于无法访问的代码和伪 API 调用的处理需要进一步改进。杨望等人^[9]提出了一种多特征集成的静态检测方法，通过提取恶意代码不同层次的特征，增强检测模型的泛化性能。基于 Bagging 与 Stacking 算法构建集成学习框架，其检测准确率达到 96.99%。Chai 等人^[10]提出了一种动态原型网络（DPNSA），能够根据样本自适应的学习，以检测恶意代码。首先，该方法将恶意代码二进制文件生成大小固定的图像，并作为嵌入模块的输入。然后，引入动态卷积神经网络，并在动态卷积中引入 SE 模块。最后，通过使用度量的方法计算恶意代码与查询样本的距离，实现恶意代码检测。Hao 等人^[11]提出了一种基于指令的内部结构特征的方法识别恶意代码。通过扩展操作码，并在此基础上使用三种无监督句子嵌入算法生成三通道恶意代码特征图片。最后使用空间金字塔池化和通道注意力模块捕获多层次指令行为语

义的图像特征。然而，该方法由于算法结构复杂，往往需要耗费大量的训练时间。同时，在特征处理方面也需要相对较长的时间，这增加了整体处理的复杂性和时间成本。Li 等人^[12]使用静态分析和统计分析的方法提取多维特征，通过 N-gram 和 TF-IDF 提取多维文本特征向量，然后通过分类器对最佳的特征进行选择，最后，使用机器学习框架对最佳特征向量以及其他特征融合进行训练。实验结果表明，使用该方法的准确率高达 98%。

（2）动态分析检测方法

动态分析方法需要在一个安全的模拟环境中运行测试程序，旨在捕获程序的实时执行行为。在此过程中，通过监控包括注册表操作、API 调用以及网络通信在内的关键活动，从而全面分析程序的行为特征，以确定该程序是否具有恶意性。

Christodorescu 等人^[13]使用数据挖掘算法从依赖图中捕获恶意程序与良性程序之间的区别。此方法能够检测出混淆的恶意代码，但是时间消耗大。Tobiyama 等人^[14]利用长短期记忆网络（LSTM）构建行为模型，将 API 转化成特征向量，并生成特征图像，然后使用卷积神经网络（CNN）执行恶意代码的检测。但是，该方法可能会由于 API 调用序列过长导致性能下降。此外，该方法使用的数据集规模过小，模型容易过拟合。Rosenberg 等人^[15]将原始 API 序列划分为几个子序列，用 RNN 分别检测每个子序列。检测过程中只要有一个子序列被标记为恶意，那么整个序列都将被报告为恶意序列。Ndibanje 等人^[16]提出了一种基于 API 序列的特征构建方法。他们首先在虚拟环境中获取动态 API 调用，对 API 序列进行相似性计算，以有效区分具有相似上下文行为。最后采用基于相似性的机器学习算法和统计方法来分析和检测恶意代码。Chen 等人^[17]使用一种基于深度神经网络（DNN）的恶意代码检测方法，用于学习参数增强的 API 调用。对于部分 API 调用，手动构建了一组经验规则，并结合统计规则来确定该参数是否为恶意。同时，使用 GSDMM 聚类算法将其余调用划分为不同类别，来获得其敏感性标签。实验设计了两组深度学习模型，分别进行训练。Dabas 等人^[18]提出了一种基于 API 调用序列的 Windows 恶意代码检测系统 MalAnalyser，该方法从 API 调用序列中提取频繁的 API 调用子序列。然后，将全局和局部最佳粒子群优化算法应用于频繁模式，识别出有助于恶意代码检测的重要特征。最后，使用遗传算法丰富频繁的恶意代码模式，这些模式可以有效地检测恶意代码。

（3）混合分析检测方法

混合分析是一种结合了静态和动态特征的恶意代码检测方法，其组合在某些情况下可以提高检测能力^[19]。静态分析能够追踪程序所有可能的执行路径，但它对加壳和混淆技术十分敏感；动态分析虽然对加壳、混淆技术不敏感，但是通常会消耗大量资源和内存空间。因此，混合分析技术将静态分析和动态分析的优势结合，对组合的特征进行分析，减少信息的丢失的同时可以更好地反映程序的

特性。

Zhao 等人^[20]提出了一种基于动态和静态特征的机器学习恶意代码检测方法。首先提取 PE 文件的字符串和动态链接库进行静态分析，同时在虚拟环境中执行程序，并使用二进制技术来提取汇编指令序列和 API 调用频率，最后使用朴素贝叶斯分类模型和支持向量机分类模型进行检测，分别获得了 95%与 94%的准确率。Han 等人^[21]系统地分析了程序中静态和动态 API 序列信息的相关性，提出了一种基于混合分析的恶意代码检测框架 MalDAE。MalDAE 从系统调用级别定义程序的语义，将静态和动态的 API 调用序列基于语义映射进行关联和融合，然后根据生成的混合 API 调用序列构造特征向量空间。最后，应用于随机森林分类器进行恶意代码分类。Jeon 等人^[22]提出了混合恶意代码检测方案 HyMalD，通过使用香农熵判断文件是否被混淆，以此作为混淆恶意代码的指示器。对于未混淆的文件，该方法首先提取操作码序列并应用 FastText 技术对其进行向量表示，最后通过双向长短期记忆网络（Bi-LSTM）模型完成静态分析。同时，对于混淆的文件进行动态分析，在执行报告中分析 API 调用及其类别、频率等信息并将其转化成 RGB 图像，最后使用空间金字塔池化网络（SPP-Net）检测混淆的恶意代码。该方法很好的减少混淆恶意代码的假阴性率。

（4）基于自然语言处理的检测方法

近年来受自然语言处理（Natural Language Processing, NLP）的启发，许多研究将自然语言处理术与深度学习知识相结合，应用于恶意代码的检测中。David 等人^[23]使用深度置信网络（DBN）通过深度降噪自动编码器实现恶意代码分类，采用了 1-gram 这一自然语言处理中常见统计手段，将沙盒生成的日志文件转换为固定长度的字符串表示。然后，他们在文件中提取出常用的单词构成字典，通过检查在每个样本文件中单词中的存在情况，从而生成对应的二进制向量。Kang 等人^[24]提出了一种基于词嵌入的 LSTM 模型，以更低的维度对操作码和 API 函数名进行分析。该模型结合了签名的方法和启发式方法的优点，对恶意代码的检测和分类进行初步评估。利用微软恶意代码分类挑战数据集，比较了使用该方法和使用 One-hot 方法的对操作码和 API 函数名进行编码的分类结果。实验结果表明，当使用该方法时，其性能相较于基于 One-hot 编码的方法提升了约 0.5%，同时在训练时间上也有显著优势，缩短了约 10 分钟。Amer 和 Zelinka^[25]采用了一种基于马尔可夫链的方法来检测恶意代码。在初始化阶段计算恶意代码中每个 API 的语义相似度，并使用 K-means 对这些 API 进行聚类。随后将 API 设置为其聚类后的簇索引，构建了一个转移矩阵序列。最终通过计算 API 序列的转移概率来确定该序列是否具有恶意性。

王兴凤等人^[26]提出了一种基于序列和卷积神经网络的恶意代码检测方法，使用 Skip-Gram 算法生成 API 序列的嵌入向量，并使用 TextCNN 训练检测模型。

然而该方法并未考虑序列的前后依赖关系。Liu 等人^[27]认为连续重复的 API 调用是不必要的，因此将收集到的 API 序列中连续出现两次以上的相同 API 删减，去除冗余信息后按照调用的执行时间对 API 序列进行排序，提取出有效的 API 序列。然后使用 Word2Vec 将每个 API 调用转化为唯一的二进制向量。最后通过对 GRU, BGRU, LSTM, BLSTM 和 SimpleRNN 的检测准确率进行综合对比，确定使用 BLSTM 神经网络实现恶意代码检测。Wang 等人^[28]提出了一个名为 SIMPLE 的新模型来识别有限样本的新恶意代码家族，有效解决了样本稀缺和动态识别问题。他们利用 N-gram 获取原始的 API 序列来增强局部特征表示，并采用 Glove 方法对嵌入矩阵进行初始化。Kakisim 等人^[29]设计了一个检测框架，该框架在操作码图上执行特征学习，利用加权随机游走的策略获得操作码序列，挖掘操作码之间的关联性和潜在规律。通过应用随机游走方法进行边和节点选择，避免了处理长操作码序列带来的计算复杂性。随后使用 Word2Vec 对操作码序列进行向量化表示，由此构造了一个由低维顺序操作码嵌入组成的向量空间，并获得了极高的准确率。

总的来说，静态分析方法虽然在一定程度上能够对恶意代码进行检测，但却容易受到加壳和混淆等方式的干扰。混合分析结合了动态与静态分析的优势，可以更好地反映程序的特性，然而该方法需要占用大量的资源和内存空间。而现有的基于动态分析方法和基于自然语言处理的研究方法主要侧重于对 API 序列及其子序列展开分析，往往忽视了 API 调用名称的结构特征，同时对于程序的运行参数信息也缺乏深入的研究。这种仅基于单一特征的分析方法存在一定的局限性，进而对准确率产生影响。因此，本文从动态分析入手，将恶意代码运行过程中的 API 行为特征作为研究对象。从两个不同的角度对恶意代码检测方法进行了探讨，以进一步提高检测的准确率和效率。

1.3 主要研究内容

新型恶意代码不断涌现，使得依赖于传统特征工程和机器学习技术的检测方法难以有效应对复杂多变的安全威胁^[30-32]。这些方法受限于单一的特征来源，同时在深层次特征挖掘、计算效率以及资源消耗等方面也影响了检测的准确性和速度。因此，探索并研究一种高效且精确的新型恶意代码检测技术显得尤为迫切。API 作为恶意代码检测的关键特性，包含了程序在实际执行阶段的行为特征信息，深入分析该信息对于提升恶意代码检测的准确性至关重要。本文旨在通过设计一种基于 API 行为特征的恶意代码检测模型，围绕运用自然语言处理技术的检测方法展开研究，以动态分析为基础，分别从不同角度处理行为特征，对恶意代码进行检测。本文的主要研究工作包括：

(1) 提出一种基于 API 调用序列的恶意代码检测方法。该方法提取恶意代码动态分析报告中的 API 调用特征进行检测。在充分学习行为序列上下文语义相关性的同时,还关注每个 API 名称结构的语义信息,将 API 名称的语义信息和统计特征相结合,以解决由单一分析方式造成的信息丢失问题。此外,设计了一种基于 CNNs-BiGRU 的深度学习模型,通过并行使用多个不同尺度的卷积核并结合双向 GRU 模型,实现在学习不同尺度的行为局部特征的同时关注全局特征,提高了检测模型的准确率。为了验证所提恶意代码检测方法的有效性以及可行性,本实验在公开的大型数据集上对其进行评估。实验结果显示,该检测模型实现了 98.28% 的检测准确率,通过与各种传统的基于机器学习和深度学习检测技术比较,证明了该方案的优越性,并且达到了最终的预期效果。

(2) 提出一种基于 API 异构参数增强的恶意代码检测方法。该方法从 API 序列和 API 参数两个角度学习恶意代码行为目的。首先,采用文档向量化技术,将长度不同的调用序列转化为固定维度的矢量表示,有效解决了因序列长度差异带来的特征向量稀疏性问题。同时,设计双向时域卷积网络以学习 API 序列语义的长距离依赖关系。其次,根据异构参数的结构功能特点设计了相应的切片划分方法,并利用轻量化模型学习参数之间的关联信息。最后,融合序列特征和参数特征,从而更全面地理解程序行为。这些改进和创新有效提高了模型的检测准确性和泛化能力,并为在网络安全领域中恶意代码检测和防范提供了一种有效的解决方案。实验结果表明,该检测模型的检测准确率高达 98.4%。同时,通过对比未使用参数特征的方法以及现有使用参数特征的其他先进方法,证明了所提方法在恶意代码参数特征提取方面更具优势,能有效提升检测准确率与性能。

1.4 论文组织结构

本文共分为五个章节,逐章递进展开描述。第一章为绪论,介绍了研究背景及意义,并对恶意代码检测的研究现状进行了梳理。第二章为恶意代码检测相关基础理论及技术,介绍了本文所涉及的关键理论技术。第三章和第四章为本文的核心内容,详细介绍了两种基于 API 行为特征的恶意代码检测技术研究。最后,第五章为总结与展望,对本文的研究工作进行全面梳理与总结。每章的具体内容安排如下:

第一章为绪论部分,首先介绍了恶意代码的研究背景和意义;然后从静态分析、动态分析、混合分析和基于自然语言处理分析四个方面介绍恶意代码检测的国内外研究现状;最后介绍了本文研究的主要内容和组织结构。

第二章为恶意代码检测相关基础理论，依次介绍了恶意代码的概念和反检测技术，阐述了恶意代码检测相关技术，最后简单介绍了本文涉及的机器学习算法和深度学习算法的理论基础。

第三章详细介绍了基于 API 调用序列的恶意代码检测方法。首先介绍了方法的总体框架，然后介绍了方法实现的具体细节，最后介绍了实验与分析，包括实验环境和数据集、评估指标以及实验结果分析。

第四章详细介绍了基于 API 异构参数增强的恶意代码检测方法。首先介绍了方法的总体框架，然后介绍了方法实现的具体细节，最后介绍了实验与分析，包括实验环境和数据集、评估指标以及实验结果分析。

第五章为总结与展望。首先对本文的研究内容和创新点进行了总结，讨论了本研究有待解决的问题，并对下一步的工作和研究进行展望。

第2章 恶意代码检测相关基础理论

本章主要探讨了恶意代码检测的相关基础理论。首先，介绍了恶意代码的概念和反检测技术；接着，详细讲述了恶意代码检测相关技术的原理与方法；最后，阐述了恶意代码检测中涉及的机器学习和深度学习算法，为后续开展研究提供了必要的理论支撑。

2.1 恶意代码相关理论

随着国家和政府对安全性的持续关注与重视，网络安全防护工作的战略地位日渐凸显。与此同时，防护技术的持续进步与强化也促使攻击手段日趋多样化。鉴于此，对恶意代码基础知识的掌握成为应对与抵御各类攻击、维系网络空间和谐稳定的必要前提。深入探究恶意代码的内在机理与行为模式，有助于精准识别潜在威胁，从而构筑坚实的安全防线，确保网络环境的稳定与安全。

2.1.1 恶意代码介绍

恶意代码，即恶意软件，是指故意设计对计算机、服务器、网络或用户造成损害的软件。这类恶意代码的目的通常涉及未经授权的恶意行为，包括窃取敏感信息、破坏系统、远程控制等。一般来讲，恶意代码包括病毒、蠕虫、勒索软件、特洛伊木马、后门、间谍软件、广告软件等，也包括以上类型的组合或变异演化产生的恶意代码变种。恶意代码传播途径多样，例如受感染的网站、电子邮件附件、恶意链接、软件安装包等，这些途径的存在使得数字信息的机密性、完整性和可用性面临着严峻的风险。因此，防范和应对恶意代码威胁对于保护计算机系统和用户信息安全至关重要。

2.1.2 恶意代码反检测技术

面对不断演变和更新的检测手段，恶意代码反检测技术应运而生。作为一种对抗性手段，反检测技术的目的在于为恶意代码提供一层额外的保护，增强其欺骗性，使其更难被安全防御系统和安全分析人员所察觉。为了实现这一目标，当前的恶意代码通常采用加壳、混淆、加密等技术来躲避检测工具的追踪和分析。

（1）加壳：恶意代码加壳是一种基于加密和压缩的技术，通过在恶意代码的可执行文件中嵌入一个外部壳，实现对恶意代码或其关键片段的加密和压缩。这种技术使得恶意代码在未经解密和脱壳的情况下，难以被安全分析工具和人员所识别和检测。如图 2.1 所示，加壳的过程对程序本身的正常运行没有影响，加壳代码通常在受保护软件运行前生效，先于程序执行。一旦保护工作完成，控制

权则交还给程序的原始执行入口点（Original Entry Point, OEP），从而确保软件能够按照正常流程运行。

（2）混淆技术：恶意代码的混淆技术旨在保持代码自身功能不变的前提下，通过改变代码结构或逻辑，以干扰分析者和检测工具发现和分析其恶意行为，从而达到隐蔽和保护自身的目的。目前主流的代码混淆技术主要有：命令插入、命令替换、重命名、程序重排、数据加密等。

命令插入是指在原始代码中插入大量冗余且无实际功能的命令，在不改变程序原有执行结果的前提下，显著增大代码的体积和复杂性。通过引入大量无关的指令，使得分析者难以理解程序的实际逻辑。

命令替换是通过将原始代码中的某些命令替换为具有相同功能的新命令来进行混淆。这种替换并不改变程序的行为，但会增加代码的多样性，以绕过传统的依靠特征码的检测方法。

重命名指将原始标识符替换为无意义的标识符，如类名、变量名、函数名等。通过修改代码中的标识符，使分析者难以分析和理解代码。

程序重排则是通过重新排列代码中程序的执行顺序，使得程序的控制流结构发生改变，增加了程序的代码路径的多样性。

（3）加密：加密技术指对恶意代码的指令、数据或整个代码段进行加密的操作。在恶意代码加密技术中，通常还包括在运行时动态解密代码的步骤。解密器的作用是解码加密的代码，使其能够在内存中正常执行。这种加密的手段有助于增加恶意代码的隐蔽性，提高其对抗检测和分析的能力。

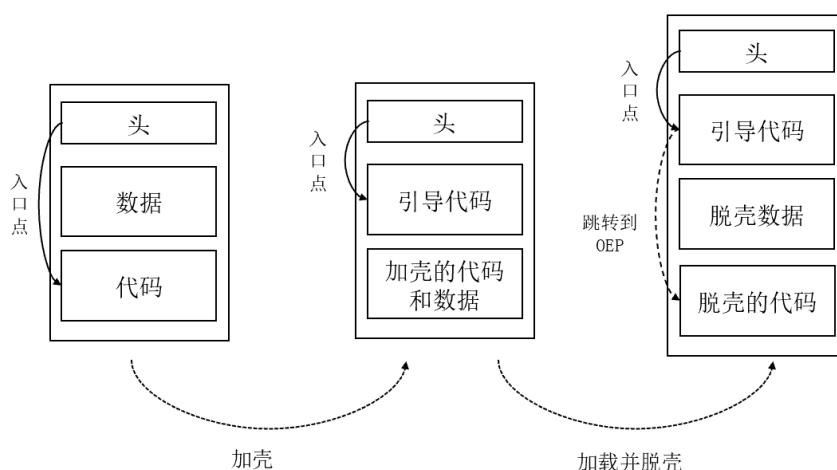


图 2.1 代码加壳及加载过程

2.2 恶意代码检测相关技术

恶意代码的广泛传播给网络安全带来了严峻的挑战，如何对这些软件进行有效检测和精准识别，已成为维护网络安全体系稳定、保障用户数据安全所必须面临和解决的紧迫任务。为此，本节将进一步介绍恶意代码检测的相关技术手段，包括：静态分析技术、动态分析技术以及结合两者优势的混合分析技术，以便更全面的认识和了解当前针对恶意代码检测方法的进展与实践现状。

2.2.1 静态分析技术

恶意代码静态分析技术是一种不依赖于程序实际执行的分析方法，它通过分析和检查恶意代码样本的源代码或二进制文件来识别其中是否包含恶意行为。该方法着重于对程序的结构、字节序列、控制流图以及操作码序列等静态属性进行深入解析。分析人员常采用 IDA Pro^[33]、Ollydbg^[34]、PEID^[35]、PEview^[36]等静态分析工具，将复杂的二进制文件转换成人类可读的汇编语言形式。借助这些先进的静态分析工具和技术，分析人员能够系统性地梳理出恶意代码的关键执行路径与恶意意图。由于静态分析并不需要执行恶意代码样本，因此能够避免潜在的系统损害风险，确保了分析过程的安全性和可靠性，并在样本处理速度和效率上表现出色。然而，恶意代码作者为了躲避安全工具检测并扰乱静态分析结果，往往采用加壳、混淆等手段来隐藏其真实内容，使分析工具难以进行有效的反汇编。恶意代码往往通过执行二进制代码来实现其设定的特定功能。

PE（Portable Executable）文件是 Windows 操作系统中使用的可执行文件或程序的标准文件格式^[37]，涵盖了多种文件类型，如可执行文件（.exe）文件、动态链接库（.dlls）以及设备驱动程序（.sys）等。其中，可执行文件作为最常用的文件格式，往往被恶意代码所利用，成为攻击和破坏的载体。PE 文件的结构相当复杂且严谨，从顶部的 DOS 头开始，依次是 PE 文件头、区块表（Section Table）、块（Section）和调试信息。DOS 头位于可执行文件的起始位置，占据 PE 文件前 64 个字节，其主要作用是为 MS-DOS 系统提供兼容性支持，确保 PE 文件在 MS-DOS 环境下也能被正确识别和加载。PE 文件头位于 DOS 头之后，它包含了 PE 装载器在加载执行文件时所需的关键字段。PE 装载器会依据 DOS 头中 IMAGE_DOS_HEADER 结构中的“e_lfanew”字段定位 PE 头的起始偏移量，并结合文件的基址获取指向 PE 文件头的指针。区块表包含了 PE 文件中各个块的描述信息，如块的名称、虚拟内存地址、物理文件偏移量、大小等，每个块对应着 PE 文件中的一个逻辑和物理上的划分单元。

2.2.2 动态分析技术

动态检测与静态检测的主要差异在于，动态检测需要实际运行恶意代码样本，

对样本在操作系统中的资源调度行为进行实时监控和详细分析。该方法可以全面了解恶意代码的实际执行情况，包括恶意代码的系统调用、文件系统、网络通信、注册表、进程和线程行为等。通过实时监测这些关键行为，动态检测能够更准确的实现对恶意代码的分类和检测。然而，动态检测也存在一定的局限性。首先，直接在实际计算机上执行恶意代码存在潜在的系统损害风险，这可能导致数据丢失、系统崩溃甚至更严重的安全问题。其次，对于大规模的恶意代码分析任务，分析人员需要不断地重装系统，这限制了批量分析的效率。为了克服这些问题，研究人员通常会采用模拟环境进行动态检测。沙箱分析工具，如 Cuckoo Sandbox^[38]、Malwr^[39]、微步云沙箱等，提供了一个受控的虚拟环境，可以有效隔离和管理恶意代码的运行，保障安全地运行恶意代码，从而避免对真实系统造成损害。这种方法不仅提高了安全性，还使得大规模分析恶意代码变得更为可行。在对恶意代码的分析过程中，观察恶意代码与操作系统间的交互行为尤为关键。恶意代码通常依赖于 Windows API 以执行特定函数调用，同时也会通过注册表来隐蔽存储和管理相关配置信息。此外，恶意代码还可能利用动态链接库来扩展其功能，或者通过统一资源定位符（URL）进行网络通信。通过对这些行为进行监测和分析，可以有效揭示恶意代码的行为模式特征，从而及时识别和阻止潜在的威胁。

本文使用的是基于沙箱的动态分析方法，为此对动态分析技术的关键特征进行详细阐述，以便更全面地理解其工作原理和应用价值。

（1）Windows API

Windows API（Application Programming Interface，应用程序编程接口），也称为 Win32，是专为 Microsoft Windows 操作系统家族设计的系统编程接口，包含了所有 API 函数的集合。在 Windows 平台上，几乎所有的程序都依赖于调用一系列 API 函数来实现各自的功能。这一特性使得 Win32 API 成为了 Windows 操作系统的核心构件，同时，恶意代码也利用这一点，将其作为侵入 Windows 平台的入口。因此，监控和分析 Windows API 调用被认为是恶意代码检测的最重要特征之一^[40-42]。通过深入研究 API 的使用模式，安全领域专家可以发现恶意代码的行为规律，进而提升对潜在风险的辨识和防御能力。

获取 API 序列的方法通常包括静态分析和动态分析两种途径，静态分析主要在代码编译阶段进行，而动态分析则是在程序实际运行时进行。静态分析在面临加密、加壳、混淆、多态等技术时，其分析过程会受到干扰而存在误差，这会导致准确率下降并增加误报率^[43]。相比之下，动态分析能够直接在实际运行时获取 API 的相关信息，不仅可以捕获到 API 的调用序列，还能追踪其种类、路径、动态链接库、URL、注册表等参数信息，可以更准确地观察恶意代码的实际执行路径和效果。

（2）注册表

注册表是操作系统中用来存储和管理系统及应用程序配置信息的一个关键数据库。注册表包含启动项、网络配置、硬件设置、环境配置等，记录了用户及安装在计算机上的应用程序的相关信息，是控制硬件、软件、用户环境和操作系统界面的数据信息文件，对于 Windows 的正常运行至关重要^[44]。注册表包括 5 个根键，其中最常用为 HKEY_CLASSES_ROOT、HKEY_CURRENT_USER 和 HKEY_LOCAL_MACHINE。具体的，HKEY_CLASSES_ROOT 包含当前计算机启动应用程序所需的全部信息，包括对象的类标识符（CLSID）、文件扩展名与文件类型、相关程序的关联信息、应用程序之间的关联和启动信息等。HKEY_CURRENT_USER 包含当前登录用户的配置信息。每个用户登录时，系统会为其创建一个独立的 HKEY_CURRENT_USER 分支，存储个性化设置、桌面设置、环境变量等用户相关的信息。HKEY_LOCAL_MACHINE 对所有用户都有效，它包含计算机的全局配置信息，包括硬件信息、操作系统信息和安全数据等。

许多恶意代码会修改注册表中的自启动项，以确保它们在系统启动时自动运行。通过添加或修改相应的注册表键值，恶意代码可以在系统启动时加载并执行其恶意操作，实现持久性感染。例如，某些病毒可能通过修改注册表路径“HKEY_LOCAL_MACHINE\SOFTWARE\MICROSOFT\WINDOWS\CURRENT_VERSION\RUN”来实现程序的自启动。通过修改这个路径下的键值，可以将恶意代码自身添加到启动列表中，确保每次系统启动时都会执行恶意代码。为了进一步保护自身，恶意代码还可能会修改注册表中路径、命名或属性等相关项，以尝试隐藏自己，避免被用户或防病毒软件检测。此外，恶意代码可能通过修改注册表中的网络代理设置，将用户的网络流量导向恶意服务器，达到窃听敏感信息、进行恶意操作或传播的目的。

（3）动态链接库

在 Windows 操作系统架构中，应用程序无法直接与硬件或底层系统资源进行交互。但是，这些应用可以通过调用一系列关键的动态链接库所封装的功能接口，来间接地实现对硬件及系统资源的访问^[45]。动态链接库是 Windows 系统中的共享函数库，这些函数可以被多个程序共享和重用，以减少磁盘空间占用、提升系统内存利用率并促进代码复用。动态链接库为用户应用程序提供了一种安全且可控的中介渠道，使它们能够在遵循安全和权限控制机制的前提下，有效且灵活地操作各种资源。然而，这一合法机制在恶意代码的操纵下存在被滥用的风险，恶意行为可能会伪装成正常的动态链接库调用，借此实现持久化驻留、注入恶意行为或扩展攻击范围等非法活动。举例来说，kernel32.dll 控制着系统的内存管理、文件操作、进程和线程管理、数据的输入输出操作和错误处理等基础的系统

级功能。kernel32.dll 的损坏或缺失，可能会导致相关应用程序无法正常运行，甚至可能影响到整个系统的稳定性。与此同时，user32.dll 作为 Windows 用户界面相关应用程序接口，负责提供用户界面（UI）相关的功能和消息处理机制。如果 user32.dll 文件丢失或损坏，可能会导致与用户界面相关的许多功能无法正常工作。再者，gdi32.dll 是 Windows 系统中的图形设备接口，它为应用程序提供了一系列用于在屏幕上绘制图形、显示和输出文本的核心功能。当 gdi32.dll 文件丢失或损坏时，依赖于图形设备接口（GDI）进行图形输出的应用程序和游戏可能会无法正常运行或显示错误信息。

（4）URL

URL（Uniform Resource Locator，统一资源定位符）是用于标识和定位互联网上资源的地址，通常由协议（例如 HTTP 或 HTTPS）、主机名、路径和查询参数组成。URL 为用户提供了一种简便的方式来访问和分享互联网上的信息和服务，用户只需在浏览器或其他支持 URL 的应用程序中输入 URL，即可与远程服务器进行通信，获取或交互所需的资源。然而，随着互联网的发展，恶意 URL 也威胁着网络安全。恶意 URL 是指向包含有害内容或者意图欺骗用户的网址。恶意 URL 可能引导用户进入网络钓鱼网站，这些网站通过模仿合法网站的外观和功能，诱导用户输入个人敏感信息，从而窃取他们的登录凭据和个人资料。此外，这些 URL 还可能链接到恶意代码的下载页面，用户一旦点击就会自动下载并安装病毒、木马、勒索软件等威胁用户计算机安全的程序^[46]。还有的恶意 URL 还会利用漏洞发起攻击，导致数据泄露或系统受损。

2.2.3 混合分析技术

随着计算机和互联网的普及，恶意代码的数量、种类和复杂性不断增加。新型的恶意代码采用了更加隐蔽、多样化的传播方式和攻击技术，使得传统的检测手段难以应对。仅依靠静态分析或动态分析往往无法充分覆盖恶意代码的各个方面。静态分析虽能深入剖析代码的结构和潜在行为，但难以捕捉其动态执行时的真实行为；而动态分析尽管能观察代码在实际运行时的行为，却可能受到实验环境复杂性和恶意代码逃避技术的影响。鉴于恶意代码的不断进化和变化，需要一种更加全面、综合的分析方法来提高检测的准确性和鲁棒性。因此，混合分析技术应运而生。混合分析技术通过将静态分析与动态分析的优势相结合，对二者的特征进行深入分析。这种方式不仅能够补充彼此的长处，更能全面、准确地反映程序的特性。在一定程度上，混合分析技术有效减少了信息丢失的可能性，从而提高了恶意代码检测的精确性和可靠性^[47,48]。

混合分析步骤通常包括：首先，对恶意代码进行静态分析，利用逆向工程等工具检索代码中的字符串、导入函数以及其他关键静态特征，以便研究其结构和

功能；接着，通过部署沙箱等动态调试和监视工具，跟踪恶意代码的执行轨迹和系统调用，以捕获其运行时行为和系统交互；最后，通过数据挖掘和关联分析，整合静态和动态分析的结果，以确认恶意代码的核心功能和目的，生成最终的检测报告。混合分析作为一种综合性研究方法，具有全面性和深入性的优点，能够结合多种方法和技术的优势，从而得出更准确、更丰富的结论。然而，混合分析的复杂性相对较高，往往需要消耗更多的计算资源和时间。特别是在处理大规模恶意代码样本集时，混合分析的效率相对较低。因此，在实际应用中，需要根据实际具体需求和资源情况，权衡各类分析技术的优劣势，科学合理地挑选出最适合的分析手段，以获取最佳的检测效果。

2.3 机器学习与深度学习算法

机器学习算法和深度学习算法是人工智能领域的两大核心技术，主要用于数据驱动的模式识别、决策和预测任务。机器学习算法基于统计学和概率论，允许计算机系统通过经验学习和改进，以便进行预测、分类或决策，包括监督学习和无监督学习等多种方法。其特点在于它们可以发现输入数据中的规律，并根据规律调整内部参数，以优化预测性能。深度学习算法是机器学习的分支，通过模仿人类大脑的结构和功能，构建深层神经网络模型。深度学习算法能够自动学习数据的层次化特征表示，相比于传统的机器学习算法，深度学习算法在处理大规模、高维度的数据上表现出色。

2.3.1 机器学习算法

机器学习是基于数据构建概率统计模型，从而对数据进行预测或决策的方法，是人工智能的一个子领域。机器学习在多个领域都展现了巨大的潜力和应用前景。传统的机器学习方法根据学习方式的不同主要分为监督学习和无监督学习两类。

（1）监督学习

监督学习（Supervised Learning）是指从带有标签的数据集中学习预测模型的问题。算法通过分析并理解每个训练样本的多个特征，掌握每个类别的显著特征，从而判断测试样本的所属类别。监督学习的本质是学习输入数据和输出类别的映射规律，构建一个能够对新的、未知类别的测试样本进行准确分类或回归预测的模型。常见的算法有朴素贝叶斯、支持向量机和决策树等。例如，在分类任务中，朴素贝叶斯算法基于属性条件独立假设，假设每个属性之间相互独立地对分类结果产生影响，以此来推断出最有可能的类别标签；支持向量机通过构建一个超平面，使得不同类别的样本中距离超平面最近的支持向量的间隔最大化，实现对样本的分类；决策树基于树结构进行决策，并根据信息增益、基尼指数等准则来选择最优分割特征，从而构建一颗具有层次性的树形模型。

监督学习利用大量带有标签的样本来建立模型，使其能够对新数据进行准确的预测。在实际应用中，监督学习广泛应用于医疗诊断、文本分类、图像识别、风险评估等领域，并展现了出色的性能，为决策制定、模式识别和智能系统的发展提供了强有力的支持。

（2）无监督学习

无监督学习（Unsupervised Learning）是指从无标签的数据中学习预测模型的问题。算法通过对输入数据的结构、模式或关系的学习来进行建模，这一学习方式摒弃了传统有监督学习中依赖于标记样本进行训练，而是探索数据内在的潜在分布和关联性。无监督学习的本质是学习输入数据的直接规律或潜在结构，以便将数据划分成不同的族群、提取出数据的主要特征成分或捕捉数据点之间的高维空间中的距离和分布等。其中，聚类算法是无监督学习中研究最多且广泛应用的一种。例如，K-means 算法通过对数据点进行迭代分组，找到数据内部自然形成的簇结构；层次聚类通过逐步合并或划分数据集中的数据点，构建出可以反映数据点间距离和亲缘关系的树状结构图。此外，还有 DBSCAN 寻找密度相连的数据区域，以及谱聚类利用图论中的拉普拉斯矩阵进行聚类等方法。

无监督学习不仅在数据挖掘、模式识别、异常检测等领域有着广泛的应用，而且对于理解大规模复杂数据集的内在本质和发现有价值知识具有极其重要的意义。无监督学习技术的深入研究和应用，使得从未知和未标注的数据中发现潜在信息成为现实，进一步提升数据分析和决策制定的能力。

2.3.2 深度学习算法

（1）卷积神经网络

卷积神经网络（Convolutional Neural Network, CNN）^[49]是一种深度前馈神经网络，其独特之处在于具有局部连接和权重共享的特性。这一设计不仅在很大程度上减少了网络中的参数数量，从而简化了网络结构，同时也增加了网络优化的可行性。此外，通过降低模型的复杂度，卷积神经网络进一步减小了过拟合的风险，提升了模型的泛化能力。

如图 2.2 所示，卷积神经网络通常由多层网络构成，其网络结构主要由输入层、卷积层、激活层、池化层和全连接层构成。其中，卷积层和池化层是所有结构中最关键的部分。卷积层是 CNN 的关键组件之一，用于图像和序列数据的特征提取。在卷积过程中，可学习的卷积核（或过滤器）与输入数据进行点积运算，最终将所有乘积结果相加，得到卷积操作的输出值。卷积核通常是一个小的二维矩阵，其大小取决于设计和任务需求，较小的卷积核通常用于捕捉细节特征，而较大的卷积核则可用于捕捉更大范围的特征。由于卷积核的大小往往远小于输入特征的大小，因此卷积运算大大降低了网络的参数数量。卷积层后面通常会设置

一个池化层。常用池化方式一般包括最大值池化（Max Pooling）和平均池化（Average Pooling），前者是计算池化窗口所覆盖的局部区域内所有元素的最大值作为最终特征，后者是计算区域内所有元素的平均值作为最终特征。通过对卷积特征图执行下采样，不仅能有效提取关键特征，减轻卷积层对位置信息的过度依赖，同时这一操作在训练过程中还起到了正则化的作用，有助于防止过拟合现象的发生。全连接层也称为密集连接层，全连接层内的每个节点与前一层的所有节点都相互连接，并对应一个权重值。经过若干层卷积和池化操作后，全连接层接收上一层输出的特征，将输入向量与权重矩阵进行矩阵乘法运算，同时加上一个偏置项，进行非线性变换。通过引入非线性激活函数，全连接层能够更灵活地处理各种非线性关系，从而提高神经网络的表达能力。

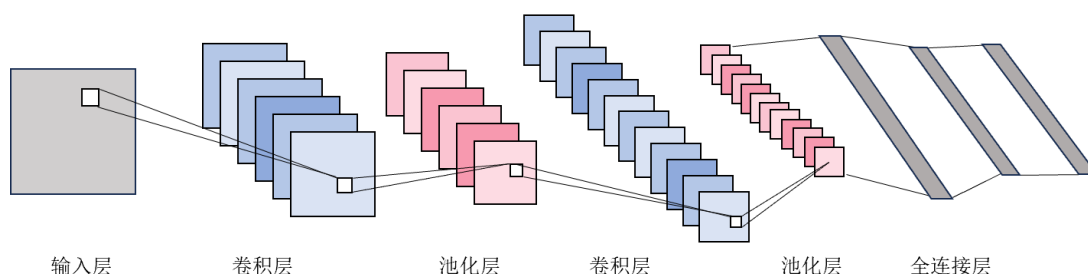


图 2.2 卷积神经网络结构示意图

（2）循环神经网络

循环神经网络（Recurrent Neural Network, RNN）^[50]是一种专门用于处理文本、信号等序列数据的深度学习模型。在传统的多层感知机（Multilayer Perceptron, MLP）^[51]和其他前馈神经网络架构中，信息流动遵循单向且静态的模式，它们在处理输入序列时无法有效地捕捉到内在的时间依赖和上下文关联性。RNN 通过引入循环结构和隐藏状态，允许信息在网络中传递，能够在处理每个新的时间步输入时不仅依赖于当前信息，还保存了前一个时间步的隐藏变量。具体地说，RNN 引入了一个新的权重参数 W_{hh} ，用于描述如何在当前时间步中使用前一个时间步的隐藏变量。无隐状态神经网络中隐藏层输出的计算方式，以及具有循环机制下隐藏状态的 RNN 模型中隐藏层在当前时间步的输出的计算方式分别可表示为：

$$H = \phi(XW_{xh} + b_h) \quad (2.1)$$

$$H_t = \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h) \quad (2.2)$$

其中， H_t 为当前时间步 t 的隐藏变量， X_t 为当前时间步的输入向量， H_{t-1} 为前一个时间步的隐藏变量， W_{xh} 为从输入到隐藏层的权重矩阵， W_{hh} 为隐藏层到隐藏层自身的循环权重矩阵， b_h 为隐藏层的偏置项， ϕ 为激活函数，例如 sigmoid、

$\tanh$ 等。

图 2.3 显示了循环神经网络在相邻时间步间信息传递的计算逻辑。具体而言，在任何给定的时间步 t ，隐状态的计算可以描述为：首先，将来自当前时间步 t 的输入数据 X_t 与前一时间步 $t-1$ 的隐状态 H_{t-1} 进行整合；随后，将两者融合的结果送入带有激活函数 ϕ 的全连接层；最终，该全连接层输出的结果即为在当前时间步 t 更新后的隐状态 H_t 。

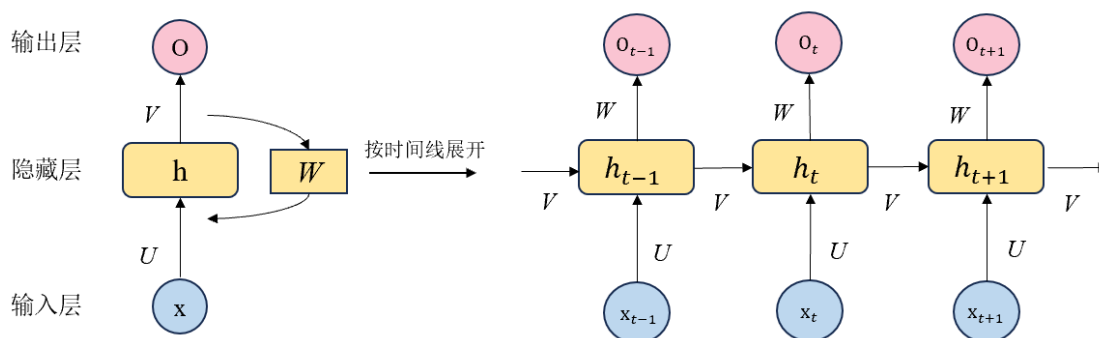


图 2.3 循环神经网络结构图

（3）长短期记忆网络

LSTM（Long Short-Term Memory, LSTM）^[52]是一种时间循环神经网络架构。它通过引入记忆元（Memory cell）、输入门（Input gate）、输出门（Output gate）和遗忘门（Forget gate）的概念，有效地捕获并整合长距离上下文信息，解决了序列数据长距离的依赖问题。在 LSTM 中，记忆元负责记录重要的上下文信息；输入门决定要在记忆元中存储当前时刻新输入数据的哪些特征信息；输出门用来决定哪些内容应作为当前时间步的隐藏状态输出，并进一步传递至下一个时间步；遗忘门用决定着要从记忆元中丢弃什么信息，负责记忆元内容的管理和重置。它们都由带有 Sigmoid 激活函数的全连接层进行处理，将这三个门的输出值控制在 0 和 1 之间。

LSTM 的结构如图 2.4 所示。假设有 h 个隐藏单元，输入为 X_t ，前一个时间步的隐状态为 H_{t-1} 。相应的时间步 t 的输入门 I_t 、遗忘门 F_t 以及输出门 O_t 的公式依次可表示为：

$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i) \quad (2.3)$$

$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f) \quad (2.4)$$

$$O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o) \quad (2.5)$$

其中， W_{xi} 、 W_{hi} 、 W_{xf} 、 W_{hf} 、 W_{xo} 、 W_{ho} 为对应的权重参数， b_i 、 b_f 、 b_o 是偏置参数。

候选记忆元是在每个时间步计算出来的一个备选的记忆元，根据当前输入 X_t 和前一时间步的隐状态 H_{t-1} 计算，这里使用 $\tanh$ 激活函数将输入的线性组合映射到 $[-1,1]$ 范围内。候选记忆元的计算可表示为：

$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c) \quad (2.6)$$

其中， W_{xc} 、 W_{hc} 为对应的权重参数， b_c 为偏置参数。

记忆元的计算过程整合了上一时间步的记忆元 C_{t-1} 和当前时间步的候选记忆元 $\tilde{C}_t$ 的信息。其中，遗忘门 F_t 控制着上一时间步的记忆元 C_{t-1} 中的信息有多少传递到当前时间步的记忆元，输入门 I_t 控制着当前时间步的输入 X_t 通过候选记忆元 $\tilde{C}_t$ 的信息是否保留。如果遗忘门的激活值接近于 1 且输入门的激活值近似于 0，则保存过去时间步的记忆元并传递到当前时间步，丢弃当前输入。记忆元的计算可表示为：

$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t \quad (2.7)$$

最后计算隐状态，通过 $\tanh$ 激活函数， H_t 的值介于-1 和 1 之间，若输出门接近于 1，则将所有记忆信息传递给预测部分，若输出门接近于 0，则不需要更新隐状态。隐状态的计算可表示为：

$$H_t = O_t \odot \tanh(C_t) \quad (2.8)$$

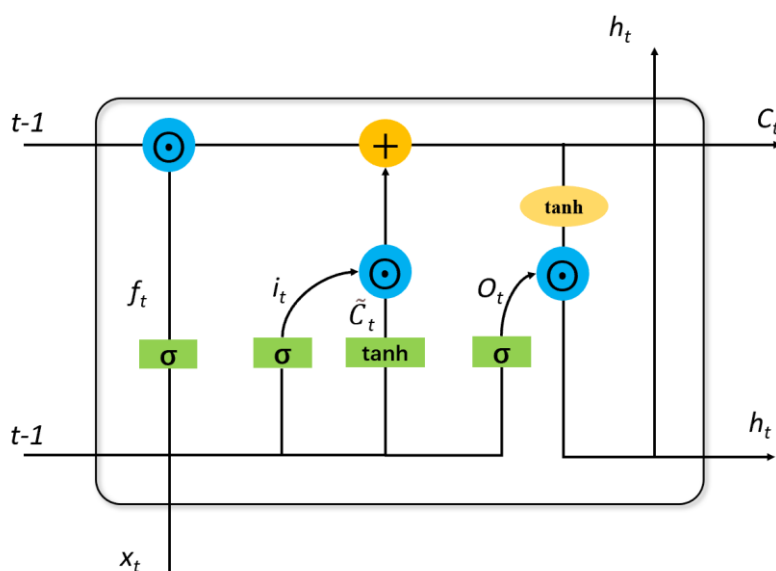


图 2.4 LSTM 循环单元结构图

(4) 门控循环单元

门控循环神经网络（Gated Recurrent Neural Network，GRU）^[53]是基于 LSTM 进行了简化与改进的神经网络模型，在保持捕捉时序依赖性的同时，显著

提升了计算效率并降低了模型复杂度。GRU 的结构如图 2.5 所示。相较于 LSTM 所采用的输入门、输出门以及遗忘门结构，GRU 通过融合这些组件，提出了两个独特的门控单元：重置门（Reset gate, r_t ）和更新门（Update gate, z_t ）。重置门主要负责控制当前时刻信息进入隐状态的程度，它决定了上一时间步的信息是否丢弃或保留，以捕获序列的短期依赖关系。而更新门则决定了前一时间步隐状态有多少比例应当被保留到当前时刻的隐状态中，以保持长期依赖性。GRU 通过当前输入 x_t 和上一个节点传递下来的隐状态 h_{t-1} 来获取两个门控的信息。重置门 r_t 和更新门 z_t 的计算过程如下：

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (2.9)$$

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (2.10)$$

这一设计使得 GRU 在保持捕捉长程依赖性的同时，简化了模型结构、减少所需的参数数量，从而有效地实现模型的轻量化。

GRU 模型在获取门控信息后，将重置门 r_t 与上一时间步的隐状态结合，得到时间步 t 的候选隐状态 $\tilde{h}_t$ 。为了确保候选隐状态中的数值能够保持在有效且易于训练的区间内，采用非线性激活函数 $\tanh$ 进行变换约束，使生成的候选隐状态中的值在保存区间 -1 到 1 内。候选隐状态 $\tilde{h}_t$ 的计算公式如下：

$$\tilde{h}_t = \tanh(W \cdot [r_t h_{t-1}, x_t]) \quad (2.11)$$

最后计算最终的隐状态，如式（2.12）所示。当更新门 z_t 接近与 1 时，则模型会倾向于只保留旧的隐状态，此时，输入 x_t 的信息就会被忽略；相反，当更新门 z_t 接近与 0 时，新的隐状态就会接近候选隐状态 $\tilde{h}_t$ 。隐状态 h_t 的计算公式如下：

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t \quad (2.12)$$

其中， W_r 、 W_z 、 W 分别表示为相对应的权重参数。

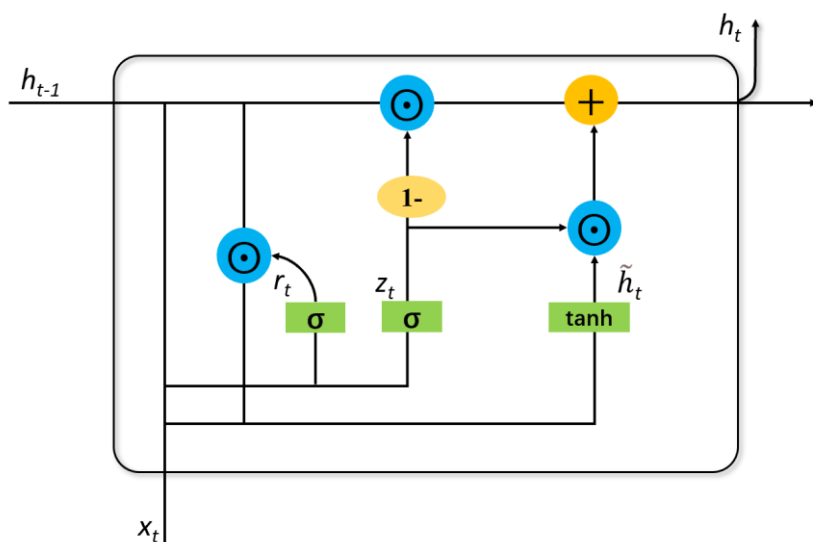


图 2.5 GRU 循环单元结构图

(5) 时域卷积网络

随着深度学习在众多领域的迅速普及，循环神经网络等各类神经网络在不同应用场景中都取得了显著的成就。然而在实际应用中，循环神经网络在处理序列数据时却存在挑战：由于 RNN 仅能处理一个时间步长，这就意味着后续步骤必须等待前一步的处理完成后才能继续执行运算。这一特性导致了 RNN 对计算资源需求较高，因为必须在整个任务完成之前保存所有中间结果。这不仅增加了计算复杂度和资源消耗，还导致了 RNN 无法像 CNN 那样实现大规模的并行处理，尤其在使用 RNN 处理长序列数据时，运算效率低下的问题变得更加显著，带来高昂的训练成本。此外，CNN 虽然在图像分类、目标检测领域有出色的表现，但由于其受限于局部感受野，难以捕捉序列数据的长距离依赖关系。为了克服这一难题，Bai 等人^[54]提出了一种用于序列建模的时间卷积网络（Temporal Convolutional Network, TCN），该模式使用了因果卷积、空洞卷积、残差连接等卷积计算方法，在学习时间数据时表现优异。

1) 因果卷积

TCN 与 CNN 的区别在于其卷积的因果关系。TCN 具有两个特点：首先，它不涉及未来到过去的信息流；其次，因果卷积层叠得越深，追溯的历史信息越久远。

在因果卷积（Causal Convolution）中， t 时刻的输出仅能使用 t 时刻及 t 之前时刻的输入计算，即 $X = \{x_1, \dots, x_{t-1}, x_t\}$ 。因果卷积的条件概率计算公式如公式（2.13）所示。若想要追溯更久远信息，就需要一个非常深的网络，模型也会变得更加复杂。为了解决这一问题，引入了空洞卷积来实现更广泛且深入的感受野。

$$p(x) = \prod_{t=1}^T P(x_t | x_1, \dots, x_{t-1}) \quad (2.13)$$

2) 空洞卷积

空洞卷积（Dilated Convolution）通过在标准的卷积里注入空洞，来有效地扩大 CNN 的感受野。这种扩展允许顶层输出能够适应更广泛的输入范围，在不改变参数数量的情况下扩展网络的感受野，降低了网络的复杂性。图 2.6 显示了卷积核 $k = 2$ 和空洞因子 $d = \{1, 2, 4, 8\}$ 时的 TCN 网络结构的示意图。空洞卷积的计算定义为：

$$F(t) = \sum_{i=0}^{k-1} f(i) \cdot x_{t-d \cdot i} \quad (2.14)$$

其中 x, t, k, d 和 $t - d \cdot i$ 分别为序列输入、序列元素、卷积核大小、空洞因子和方向。

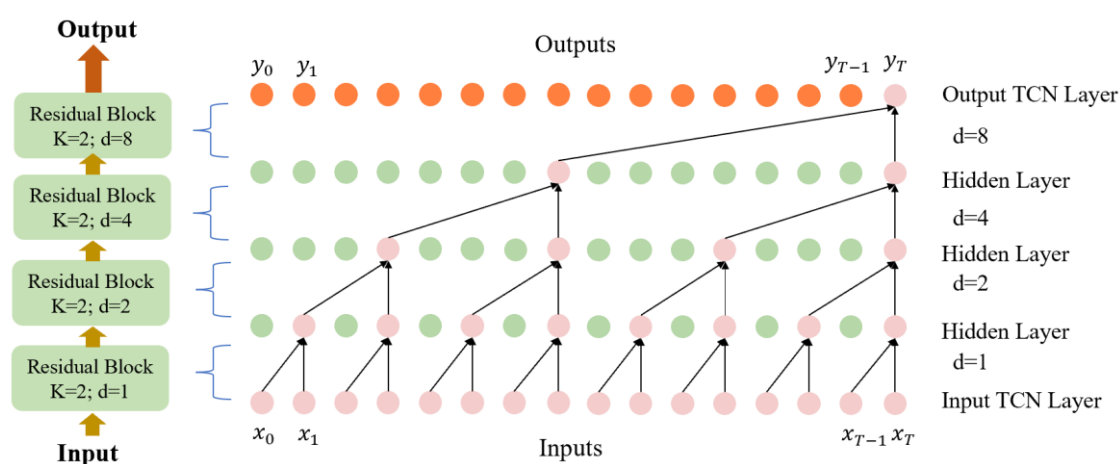


图 2.6 TCN 网络结构图

3) 残差连接

深层的卷积神经网络可以捕获不同层次的特征，更好的拟合数据，这一点在许多任务中都得到了验证。然而，在训练过程中，模型的效果不会随着网络的加深而不断提高，使用过深的网络可能会影响模型的稳定性，并可能导致梯度消失或梯度爆炸等问题。因此，为了需要构建深层的网络模型，TCN 引入了残差连接来提高模型的训练性能，如图 2.7 所示。残差块中的跳跃连接有助于保持梯度范数，从而实现稳定的反向传播^[55]。

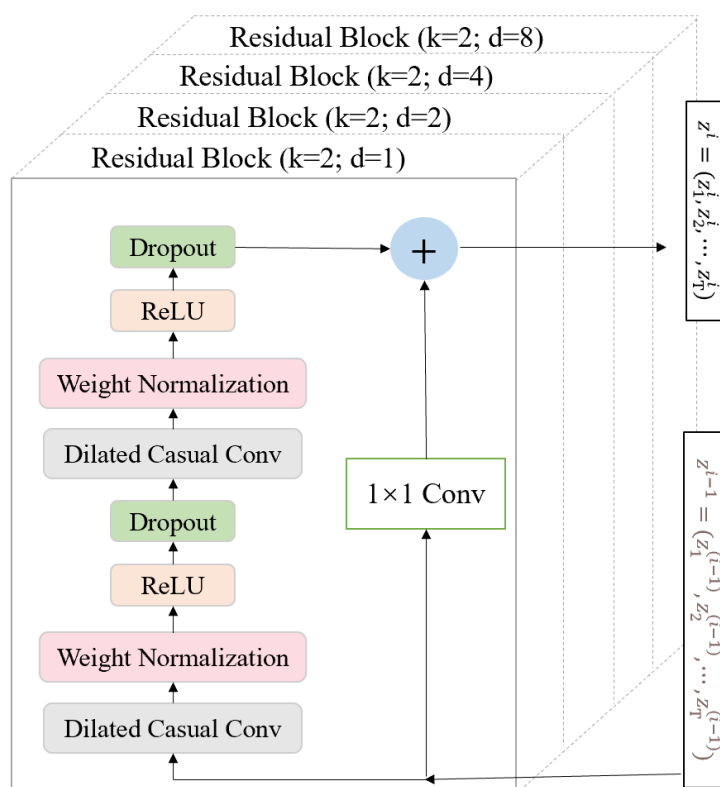


图 2.7 TCN 残差块结构图

每个残差块由两层组成：空洞因果卷积和 ReLU 激活。通过在空洞因果卷积后加入权重归一化层和 Dropout 层，进一步提高了网络的泛化性。在残差块内，一个分支对输入 x^{h-1} 进行卷积等操作，同时，引入了另一个分支执行一个简单的 1×1 卷积变换，以确保在与现有分支并行操作的同时特征映射的数量保持一致。第 h 个残差块的输出表示为：

$$x^h = \delta(F(x^{h-1}) + x^{h-1}) \quad (2.15)$$

其中， δ 和 F 分别表示激活操作和残差部分。

2.4 本章小结

本章介绍了恶意代码检测相关基础理论及技术，首先，对恶意代码的基本概念进行了介绍，并详细阐述了恶意代码为规避检测而发展的反检测策略。然后介绍了用于恶意代码检测的三种核心分析技术，即静态分析、动态分析以及混合分析技术，对各类分析技术的核心原理、优势与局限性以及关键特征进行了详尽阐述。最后本章对机器学习算法与深度学习算法的基础理论进行了介绍。本章所探讨的相关理论和技术构成了本文研究的核心和重要基础，对于实施恶意代码检测起到了指导性的作用。

第3章 基于 API 调用序列的恶意代码检测方法

3.1 引言

在当今时代，新型基础设施的崛起正在引领技术创新的浪潮。这些先进技术的广泛应用使得网络生态变得更为复杂，同时也为恶意代码的潜在威胁提供了更多的发展空间^[56]。在这样的背景下，恶意代码检测显得尤为关键。一些研究工作通过挖掘关联规则来识别恶意代码。然而，传统检测方法需要大量样本分析和规则提取的专家经验，并且需要不断更新特征库，在进行大批量恶意代码检测时人力资源和计算资源消耗大，这对检测效率造成了限制。同时，静态分析在面对零日攻击、混淆、加壳、多态或代码变形的恶意代码时表现出一定的局限性，对于恶意代码变体检测效果有限且容易受到破坏^[57,58]。

研究恶意代码的执行行为，尤其是 API 调用序列，已被广泛证实为一种对恶意代码进行检测的有效方法。现有基于动态的检测方法利用 API 调用的特征，如出现频率和序列编码等，以实现恶意代码的识别。然而，仅仅依赖 API 调用的行为模式和频次特征进行恶意代码检测，效果并不理想。这是因为恶意代码的开发者为了规避现有的检测手段，往往会故意添加多个冗余的 API 调用，使得这些特征在检测过程中失去应有的准确性，从而影响了检测的整体效果。与此同时，越来越多的研究倾向于将 API 调用序列视为自然语言处理中的序列分类问题，并选用相应的 NLP 模型进行分析与建模。但大多数现有研究仅侧重于对整个调用序列中固定长度窗口内的上下文特征进行挖掘，一定程度上忽视了对单个 API 调用的结构语义特征和统计特征的分析。

针对上述问题，本章设计并实现了一种结合自然语言处理和深度学习的恶意代码检测方法。一方面，本章将 API 调用的语义信息和统计特征相融合，在充分学习序列上下文行为的语义相关性的同时，关注每个 API 名称的结构语义信息和统计特征，从而对 API 调用进行多角度分析，提高了模型的准确率。另一方面，设计了一种融合多尺度卷积和 BiGRU 模型的架构，以捕捉 API 调用序列中不同长度的模式特征，增强了模型对序列局部和全局特征的提取能力，提高恶意代码识别的准确性和可靠性。

3.2 方法框架

本节将详细介绍基于 API 调用序列的恶意代码检测方法，总体框架结构如

图 3.1 所示。

该框架主要包括三个模块：数据预处理、特征提取和生成以及 CNNs-BiGRU 模型。首先，提取所需 API 调用序列，并通过预处理去除冗余调用。然后，基于 API 调用名称的结构特性对其重新描述，并用词嵌入技术对 API 调用序列及重新描述后的 API 名称进行向量化表示，同时结合调用名称的统计特征，丰富向量表示的信息。最后，通过融合多尺度卷积与双向门控循环单元，设计 CNNs-BiGRU 模型，以实现对于恶意代码更为精确且全面的检测。

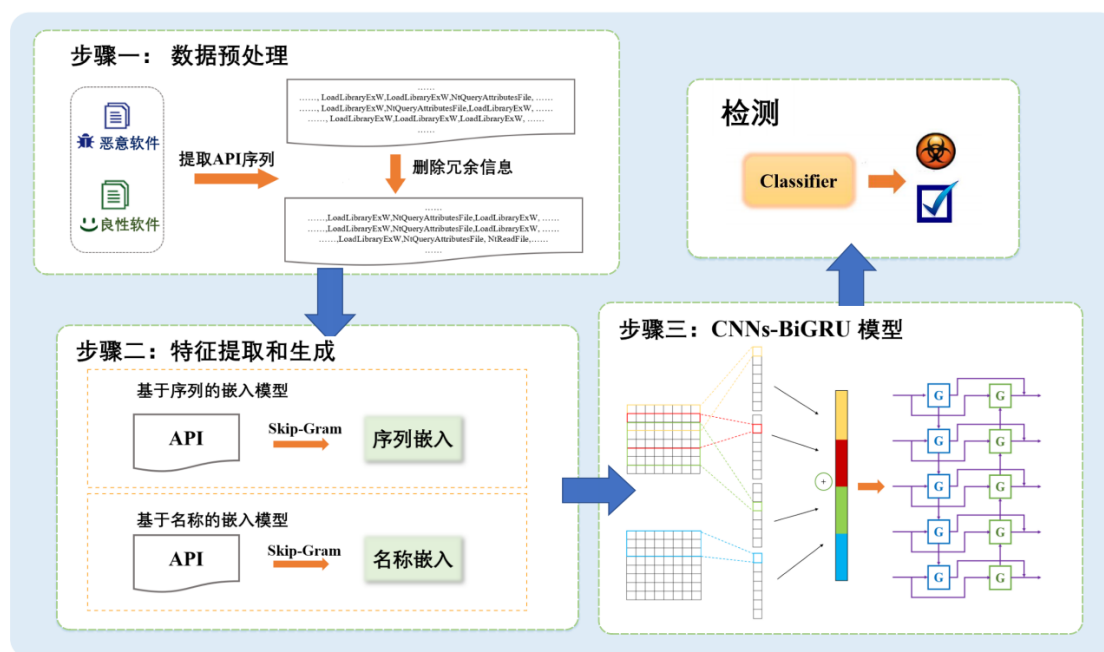


图 3.1 基于 API 调用序列的恶意代码检测的总体框架结构

3.3 数据预处理

首先从数据集中提取出 API 调用序列。在获得 API 序列后观察到有许多连续重复的 API，这是因为一个程序在执行一些与文件相关的工作或执行循环时，通常会重复调用一些相同的 API。本研究将这些相同的调用子序列视为冗余信息，并对其进行预处理，通过删除连续重复的 API 调用，仅保留最有区别性的特征，从而提高模型性能。由于卷积神经网络要求输入矩阵具有统一的维度，而每个样本中的 API 序列长度不同，因此这里对所有样本的 API 序列长度进行统一处理。具体来说，本节设定一个固定的长度标准，对于超过该长度的 API 序列进行截断，对长度不足的序列则进行补零操作。这样的处理方式确保了输入矩阵的一致性，为后续的模型训练提供了统一的输入形式，进而使得卷积神经网络能够高效处理不同长度的 API 调用序列。

3.4 序列特征嵌入

词嵌入^[59]，是一种将单词映射到 n 维向量空间的技术。在恶意代码执行的过程中，恶意代码中 API 函数的调用顺序并非随意排列，而是反映着恶意行为的潜在上下文执行模式。在不同恶意代码样本的序列结构中，这些模式往往呈现出某种程度上的共性和规律性。通过深入挖掘大量恶意代码序列中的行为模式，可以有效地揭示并理解序列内各个 API 函数之间的复杂上下文关系。

由于传统的统计方法在分析软件代码序列中的 API 调用关系时存在局限性，无法有效地揭示并捕获这些调用之间潜在的深层次上下文关联。受到自然语言处理中词嵌入技术的启发，不同于常规的随机初始化方式，本节使用预先训练好的词嵌入模型来初始化嵌入矩阵，以赋予每个 API 调用丰富的语义含义。词嵌入技术领域涵盖了多种模型，其中 Word2Vec 模型被认为是最常用的单词嵌入形式之一^[60-62]。该模型通过量化并捕捉词汇间复杂的语义关系，学习单词的分布式编码，使得在高维向量空间中，语义相似的单词能够映射至彼此距离相近的向量点。Word2Vec 包含 CBOW 和 Skip-Gram 两种模型。CBOW 模型的核心思想是利用一个关键词的前后文预测该关键词出现的概率，目标公式如式（3.1）所示。Skip-Gram 模型则正好相反，是利用一个关键词来预测该关键词前后文出现的词的概率，模型的目标公式如式（3.2）所示。与 CBOW 模型不同的是，其隐藏层到输出层的概率分布将会输出多个。

$$P(\omega_c | \omega_{o_1}, \dots, \omega_{o_{2m}}) = \frac{\exp(\frac{1}{2m} u_c^T (v_{o_1} + \dots + v_{o_{2m}}))}{\sum_{i \in V} \exp(\frac{1}{2m} u_i^T (v_{o_1} + \dots + v_{o_{2m}}))} \quad (3.1)$$

$$P(\omega_o | \omega_c) = \frac{\exp(u_o^T v_c)}{\sum_{i \in V} \exp(u_i^T v_c)} \quad (3.2)$$

其中， ω_c 和 ω_o 分别表示所选中心词和中心词的上下文词，上下文窗口大小为 m ， u_c 和 u_o 分别代表中心词向量和上下文词的词向量，全体序列词库用 V 表示。

本节使用 Skip-Gram 模型来捕获特征之间的上下文的联系，实现对 API 序列模式的有效建模。Skip-Gram 是一种基于单层神经网络的浅层架构，如图 3.2 所示。该模型将数据集中所有的 API 调用序列视为文本语料库，通过目标词来预测目标词上下文窗口中左右两侧各 C 个单词，从而学习并赋予每个 API 相应的编码表示。

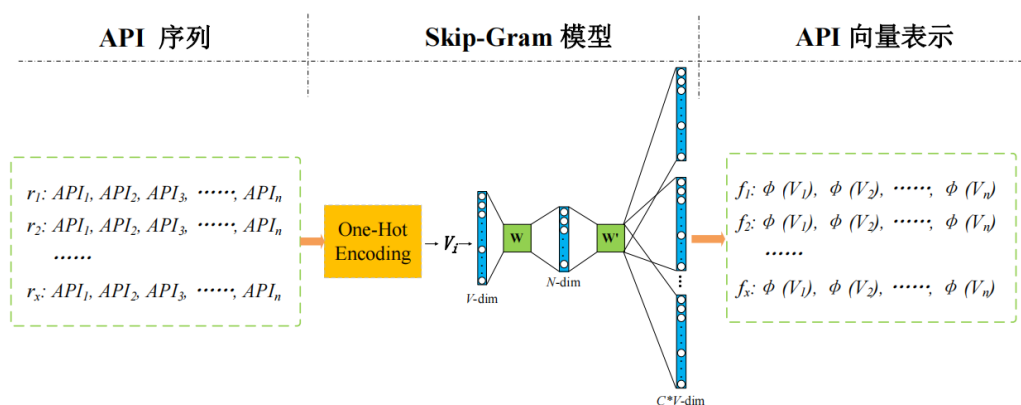


图 3.2 基于 API 序列的词嵌入模型

首先，使用 One-Hot 编码方式将每个 API 调用转化为向量表示，其中向量的维度 V 的值为语料库中统计到的唯一 API 个数。在这一过程中，对于词汇表中任一特定 API 调用，其在词汇表中的索引位置对应的向量元素设置为 1，其余所有元素则填充为 0，以实现 API 调用离散化表示。接着训练嵌入模型，根据索引映射，将每个单词从原始的离散空间映射至 N 维连续向量空间，这样所有的 API 调用都被映射到一个形状为 $V \times N$ 的矩阵 W 上，其中矩阵中每一行都对应一个调用的输入词向量。然后，通过将该 N 维向量与隐藏层矩阵 W' (W' 形状为 $N \times V$) 相乘得到多项式分布，其中矩阵 W' 中每一列对应一个调用的输出词向量。最终，为了将输出转化为概率分布形式，进一步应用 softmax 函数处理，得到每个 API 调用在该目标词前后出现的概率。训练完成后，得到嵌入矩阵 ϕ ，即第一个权重矩阵 W ，其中每一行为对应调用的词向量。这样，出现在 API 调用序列 r 中的每个 $APIv_i \in V$ 都被它自己的低维向量 $\phi(v_i)$ 所取代。而对于词汇外 (out-of-vocabulary, OOV) 词以 '0' 作为填充。

3.5 名称特征嵌入

由于操作系统开发人员通常采用基于功能语义的命名方法来为 API 进行命名^[63]，这使得每个 API 名称字符串都包含了一定的功能信息。在本节基于 API 名称定义了一种编码方式，以实现 API 名称特征的有效表示。

Li 等人^[64]提出了一种描述 API 名称信息的方法，他设计了一个动作字典，旨在从每个 API 调用中抽取出其执行的操作部分。同时，将提取操作后调用名称的剩余部分作为该操作所作用的对象。本节基于 Li 的特征表示方式来重新描述 API，并进行进一步的改进。通过分析 API 名称的构成，从中提取能够体现出 API 操作和操作对象的字符串。接着，根据 Cuckoo 沙箱提供的 API 分类标准，进一步获取了每个 API 调用的类别信息，作为名称特征的补充。具体的，

Cuckoo 沙箱共跟踪了 312 个 API 调用，涵盖了 17 个不同的类别。本节在此基础上增加了一个“other”类别，用于涵盖那些未明确分类的调用，最终将 API 调用归为共计 18 个类别。最后，按照每个 API 名称的操作、对象、类别的顺序，将其形成一个全新的序列。该新序列将作为输入数据，采用与上节中基于 API 序列嵌入相同的思路，利用 Skip-Gram 模型来生成 API 名称的特征表示。因此，在序列中表现出相似或相关行为的 API，其名称嵌入将会非常接近。

TF-IDF 是文本挖掘中常用的加权技术，本节进一步分析了 API 的 TF-IDF 值，以此统计 API 调用在恶意和良性样本中的重要程度。若该 API 在恶意样本中频繁出现（即 TF 值高），同时在良性样本中较为罕见，则认为此调用具有很好的类别区分能力。TF-IDF 的计算过程如下所示：

$$TF-IDF = TF \times IDF = \frac{n_{i,j}}{\sum_k n_{k,j}} \cdot \log \frac{|D|}{|\{j:t_i \in d_j\}|} \quad (3.3)$$

其中， $n_{i,j}$ 表示该调用在序列 d_j 中出现的次数， $n_{k,j}$ 表示序列 d_j 中所有调用出现的次数总和， D 为语料库中序列的总数， j 为包含该调用的序列数。

将由词嵌入模型获取的 API 名称的词向量与其统计特征相组合，共同实现对 API 名称的向量化表示。

最后，序列特征嵌入和名称特征嵌入共同构建出完整的 API 嵌入，如图 3.3 所示。

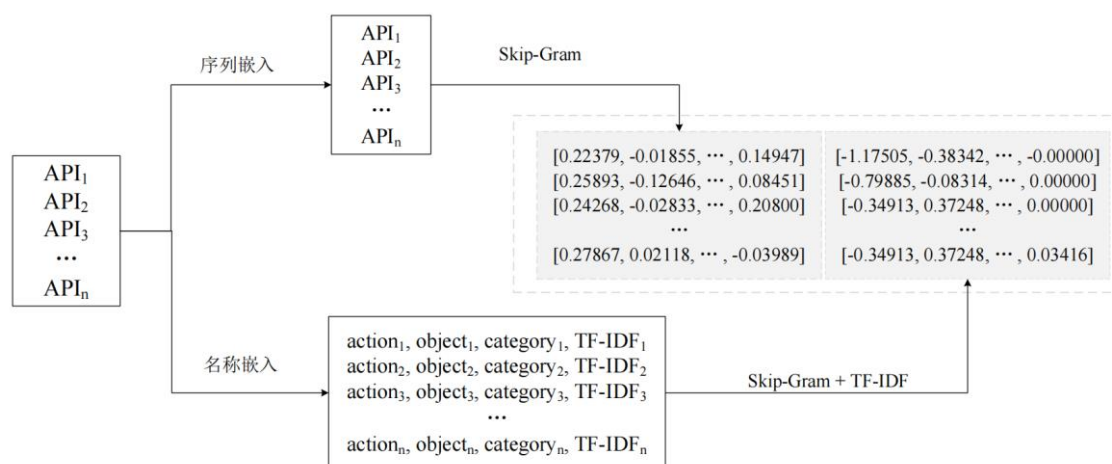


图 3.3 API 嵌入示意图

3.6 CNNs-BiGRU 模型

本研究设计了 CNNs-BiGRU 模型，该模型融合了卷积神经网络与双向门控循环单元，所提出用于恶意代码检测的 CNNs-BiGRU 每一层架构如图 3.4 所示。

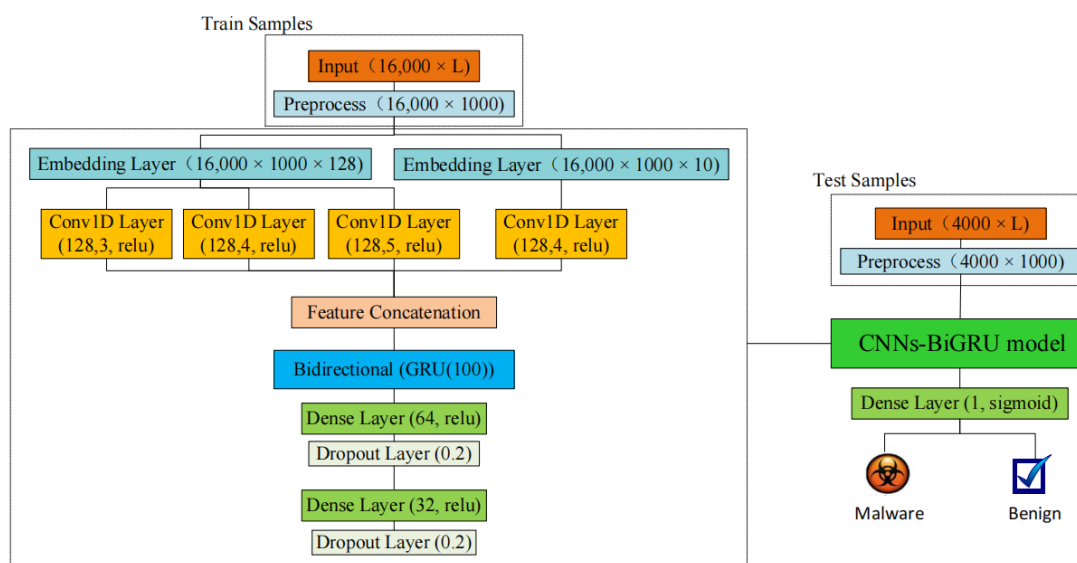


图 3.4 模型架构图

输入层：该层包含用于训练和测试的恶意代码或良性代码样本，以长度在 0 到 L 之间分布的 API 调用序列作为输入数据。本研究以随机方式选择数据集中 80% 作为训练集，剩下的 20% 作为测试集，总数据量为 20,000。预处理模块作为输入层重要组成部分，负责对所有样本的序列长度进行统一处理。这里，本研究将序列长度设置为固定值 1000，以提高性能并减少训练时间，同时最大限度地包含有用数据。

嵌入层：嵌入层将输入的 API 调用序列转化为其对应的向量表示。该过程根据 API 序列词向量模型和 API 名称特征的向量模型来确定嵌入向量，输入维度分别为 200 和 10，依次表示序列特征嵌入和名称特征嵌入。

一维卷积层：通过并行使用了多个不同尺寸卷积核来处理输入数据，以捕捉多种尺度的信息特征。具体来说，针对 API 序列特征嵌入，本方法设置了三个一维卷积层，分别采用大小为 3、4 和 5 的滤波器，以便提取不同尺度 API 调用的依赖关系。与此同时，针对 API 名称特征嵌入，应用一个滤波器大小为 4 的一维卷积层进一步提取名称特征中的局部关联信息。在每个卷积层后均采用了“ReLU”激活函数，其目的在于增强模型的学习表达能力，从而提升对潜在恶意行为特征的有效识别与检测性能。

双向 GRU 层：在拼接所有卷积层的输出后，应用 BiGRU 模块处理正向和反向的数据，以捕获 API 调用之间的前后依赖关系。BiGRU 由两个用于处理顺序数据的 GRU 组成，其中一个向前接收输入序列，另一个 GRU 向后接收输入序列，如图 3.5 所示。BiGRU 通过门控机制控制信息流，相比于传统的 BiLSTM，其设计减少了参数量，有效降低模型复杂度，有利于减少训练时间和资源消耗。

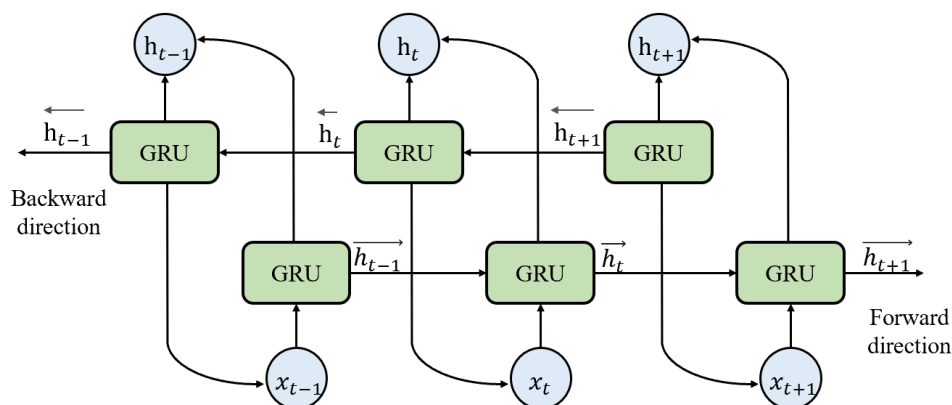


图 3.5 BiGRU 结构图

密集层：通过使用密集层，对特征空间进行逐步降维处理，首先缩减至 64 维，然后进一步压缩至 32 维。随后，应用非线性的“ReLU”激活函数，以缓解梯度消失问题。

Dropout 层：Dropout 层通过以一定概率随机停用一些神经元来有效防止模型对训练数据产生过拟合现象。在模型构建中，为了进一步缓解过拟合问题，本节引入了多个 Dropout 层。具体来说，每一个 Dropout 层都会以 20% 的概率丢弃前一层输出的神经元，以此来增强模型的稳定性和鲁棒性。

根据分类任务的需求，本方法选择 Sigmoid 激活函数作为最后一个全连接层的激活函数，并将输出维度设定为 2 维。对给定样本是否属于恶意代码进行二元概率预测，输出向量的两个分量分别对应良性与恶意两类标签的概率值，从而实现未知样本属性的有效判定。

3.7 实验与分析

3.7.1 实验环境与数据

为了对本实验进行评估，本节采用了一个通过沙盒生成的，并在第三方提供的公开数据集^[65]。该数据集于 2019 年公开，所使用的数据集包含总共 20,000 个 Windows PE 文件的执行报告，其中 10,000 个为良性的，其余为恶意的。图 3.6 显示了可执行文件在沙箱环境中运行生成的 XML 格式的报告文件片段。

```

▼<action api_name="LoadLibraryExW" call_name="l.exe" call_pid="396" call_time="18:45:27.154" ret_value="0">
  ▼<apiArg_list count="3">
    <apiArg value="0"/>
    <apiArg value="0"/>
    <apiArg value="0"/>
  </apiArg_list>
  ▼<exInfo_list count="1">
    <exInfo value="C:\WINDOWS\system32\icmp.dll"/>
  </exInfo_list>
</action>
▼<action api_name="NtQueryAttributesFile" call_name="l.exe" call_pid="396" call_time="18:45:27.156" ret_value="c0000034">
  ▼<apiArg_list count="2">
    <apiArg value="12f974"/>
    <apiArg value="12f94c"/>
  </apiArg_list>
  ▼<exInfo_list count="1">
    <exInfo value="C:\program\iphlpapi.dll"/>
  </exInfo_list>
</action>
▼<action api_name="LoadLibraryExW" call_name="l.exe" call_pid="396" call_time="18:45:27.159" ret_value="0">
  ▼<apiArg_list count="3">
    <apiArg value="0"/>
    <apiArg value="0"/>
    <apiArg value="0"/>
  </apiArg_list>
  ▼<exInfo_list count="1">
    <exInfo value="C:\WINDOWS\system32\iphlpapi.dll"/>
  </exInfo_list>
</action>

```

图 3.6 样本示例片段

图 3.7 显示了所有数据集中每个样本中 API 序列长度的分布情况，观察到大约 60% 文件的 API 序列长度小于 1000，而其余长度的文件分布在一个广泛的范围内。由于随着 API 序列长度的增加，训练过程中可能出现内存溢出的情况，因此本研究将 API 序列的最大长度限制为 1000，以有效处理不同长度的文件并避免潜在的内存溢出问题。

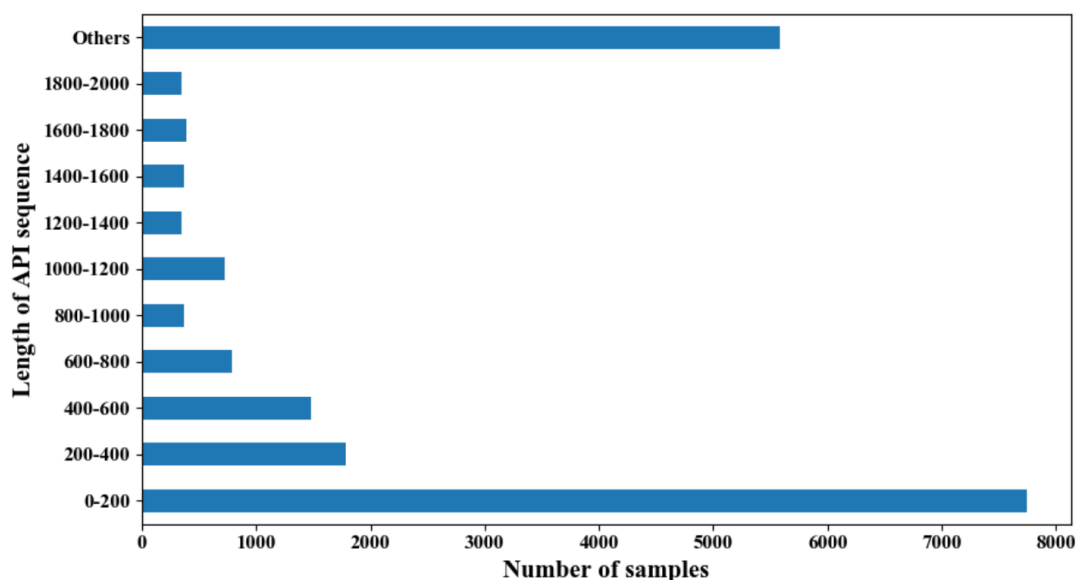


图 3.7 API 序列长度的分布情况

所提模型使用 python 语言编写实验程序，使用 pytorch 1.10.0 作为后端。实验选择了 80%的数据作为训练集，同时保留剩余的 20%用于测试。实验中的超参数设置如表 3.1 所示。实验 Batch Size 为 64，训练周期为 80。

表 3.1 实验超参数

实验超参数	实验数值
Skip-Gram 模型窗口大小	5
Skip-Gram 模型最小计数	3
基于序列模型嵌入单元	200
基于名称模型嵌入单元	10
Bi-GRU 层数	1
Bi-GRU 层单元	100
Dense 层 1	64
Dense 层 2	32
激活函数	Relu
Dropout 率	0.2
学习率	10^{-3}

3.7.2 评价方法与指标

在评估恶意代码检测模型的性能时，本章主要依赖以下四个关键指标：准确率、精确度、召回率以及 F1 分数（F1-Score）。这些指标均基于真正例（True Positives）、真负例（True Negatives）、假正例（False Positives）和假负例（False Negatives）这四个基础概念进行计算。

准确率反映了模型对所有样本的正确分类比例，准确率越高，说明模型对恶意代码和良性代码的区分能力越强。准确率公式如下所示：

$$Accuracy = \frac{True\ Positives + True\ Negatives}{True\ Positives + False\ Positives + True\ Negatives + False\ Negatives} \quad (3.4)$$

精确度衡量的是模型预测为恶意的样本中，实际为恶意的样本所占的比例，精确度越高，意味着模型预测为恶意的样本中，真正是恶意的比例越高，即误报率越低。精确度公式如下所示：

$$Precision = \frac{True\ Positives}{True\ Positives + False\ Positives} \quad (3.5)$$

召回率反映了模型在所有恶意样本中正确识别出恶意样本的比例，召回率越

高,说明模型遗漏的恶意代码越少,即模型的检测能力越强,召回率公式如下所示:

$$Recall = \frac{True\ Positives}{True\ Positives + False\ Negatives} \quad (3.6)$$

F1-Score 是精确度和召回率的调和平均值, F1-Score 越高,说明模型在精确度和召回率上都表现较好,即模型在减少误报和漏报方面都有较好的性能。F1-Score 公式如下所示:

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (3.7)$$

3.7.3 实验结果分析

(1) 不同词嵌入模型的性能比较

本实验与不同词嵌入模型进行比较,并训练不同大小的嵌入向量。如表 3.2 所示,本节应用了两种常见的词嵌入模型:Skip-Gram 和 CBOW,同时设定嵌入向量的大小为 100、200 和 128。通过对这些组合进行训练,在准确率、精确度、召回率和 F1 分数这四个方面进行了详细的比较分析。从表格数据中可以观察到,使用 Skip-Gram 模型的检测准确率始终高于使用 CBOW 模型的准确率。这是由于 Skip-Gram 注重 API 调用序列上下文中每个调用与目标调用之间的关系,生成的向量更能捕捉到调用之间的细致语义特征。此外,值得注意的是,当嵌入向量大小设置为 128 时,检测准确率和 F1 分数均达到最高点,进一步突显了选择 128 作为嵌入向量维度的优越性。

表 3.2 不同词嵌入模型的性能比较

嵌入模型	向量大小	Accuracy	Precision	Recall	F1-Score
CBOW	100	0.9770	0.9793	0.9743	0.9780
	128	0.9720	0.9724	0.9715	0.9719
	200	0.9715	0.9709	0.9719	0.9714
Skip-Gram	100	0.9775	0.9709	0.9719	0.9714
	128	0.9828	0.9870	0.9789	0.9827
	200	0.9755	0.9827	0.9680	0.9754

(2) 不同算法模型的性能比较

本实验比较了三种传统机器学习算法的性能即朴素贝叶斯 (NB)、K 近邻 (KNN) 和决策树 (DT),以及三种深度学习算法包括 BiLSTM、Text-CNN 和 BiGRU,所有算法模型都使用准确率、精确度、召回率和 F1-Score 指标来衡量

效果。

表 3.3 机器学习和深度学习模型的性能比较

	Accuracy	Precision	Recall	F1-Score
Naive Bayes (NB)	0.8198	0.8518	0.7768	0.8126
K-Nearest Neighbor (KNN)	0.9175	0.9556	0.8767	0.9145
Decision Tree (DT)	0.9178	0.9184	0.9180	0.9182
Text-CNN	0.9443	0.9587	0.9298	0.9440
BiLSTM	0.9743	0.9766	0.9723	0.9758
Proposed Model	0.9828	0.9870	0.9789	0.9827

如表 3.3 所示，深度学习模型的性能普遍优于机器学习模型，其性能提升幅度达到了 3%以上。在所有机器学习和深度学习模型中，本章提出的模型在测试数据集上取得了显著的成绩，准确率为 0.9828，精确度为 0.9870，召回率为 0.9789，F1 分数为 0.9827。这表明多个 CNN 架构能够有效捕捉不同尺度下的局部特征，有助于模型理解输入数据中的层次结构和细节。此外，BiGRU 模型则能更有效地利用上下文信息，这使得模型能够更全面地理解序列中的关系和模式，从而提高了模型的分类性能。总的来说，深度学习方法在恶意代码检测中展现出显著优势，尤其在处理复杂语义和上下文信息时。其模型不仅准确率高，而且在综合性能上也有大幅提升。

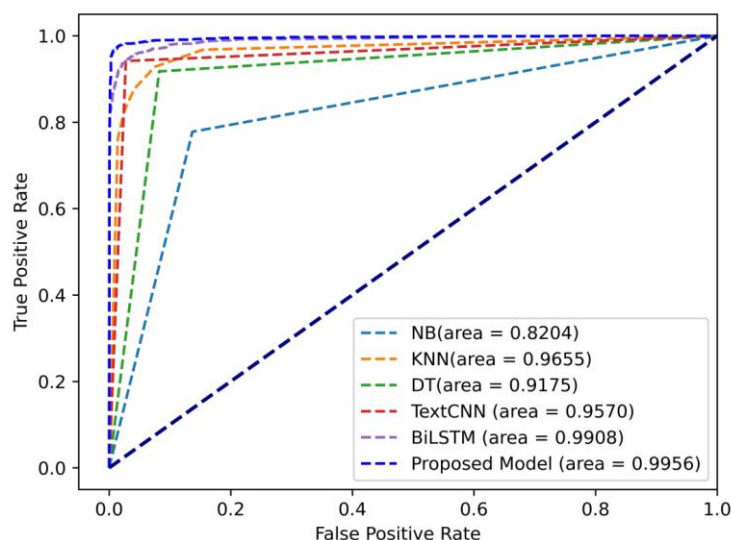


图 3.8 不同算法模型的 ROC 曲线

图 3.8 绘制了测试数据集上不同算法模型的 ROC^[66]曲线，X 轴表示假阳性率（FPR），Y 轴表示真阳性率（TPR）。其中 ROC 曲线下的面积定义为 AUC，其值越大，即曲线越接近左上角，代表该分类器效果越好。可以看出本章所提模

型在所有对比模型中取得了最佳的 AUC 分数,表明本章模型在识别恶意代码与良性代码时,不仅能够更准确地识别出真正的恶意代码,同时还能更有效地避免将良性代码误判为恶意代码。

(3) 不同特征处理模型的性能比较

为了深入分析不同特征对检测效果的影响,本节进行了一系列实验分别考察了仅具有 API 调用序列的上下文语义信息特征的 API 序列检测模型、仅具有 API 调用名称的语义结构特征和统计特征的 API 名称检测模型,以及同时包含这两种特征的检测模型。

如图 3.9 所示,实验结果表明,检测模型缺少任何一部分特征都会导致性能下降,这验证了本研究所处理的每个特征都对检测效果有着显著的影响。值得注意的是,缺少 API 序列语义信息的模块表现出最差的性能。这突显了 API 调用上下文的语义信息对于恶意代码检测的重要性,因为 API 调用并非孤立存在,其在序列中的上下文语义对于准确判断代码的恶意性至关重要。在综合性能的考量下,同时包含序列特征和名称特征的模型实现了最佳性能。尤其是在准确率和 F1-Score 这两个关键评价指标上,本章所提模型在经过 10 个训练周期左右便开始收敛,性能逐渐趋于稳定,明显优于其他两组实验。这充分证明了,综合特征的使用有助于模型更好地区分恶意代码和良性代码的行为模式,从而提高恶意代码检测的精度和鲁棒性。

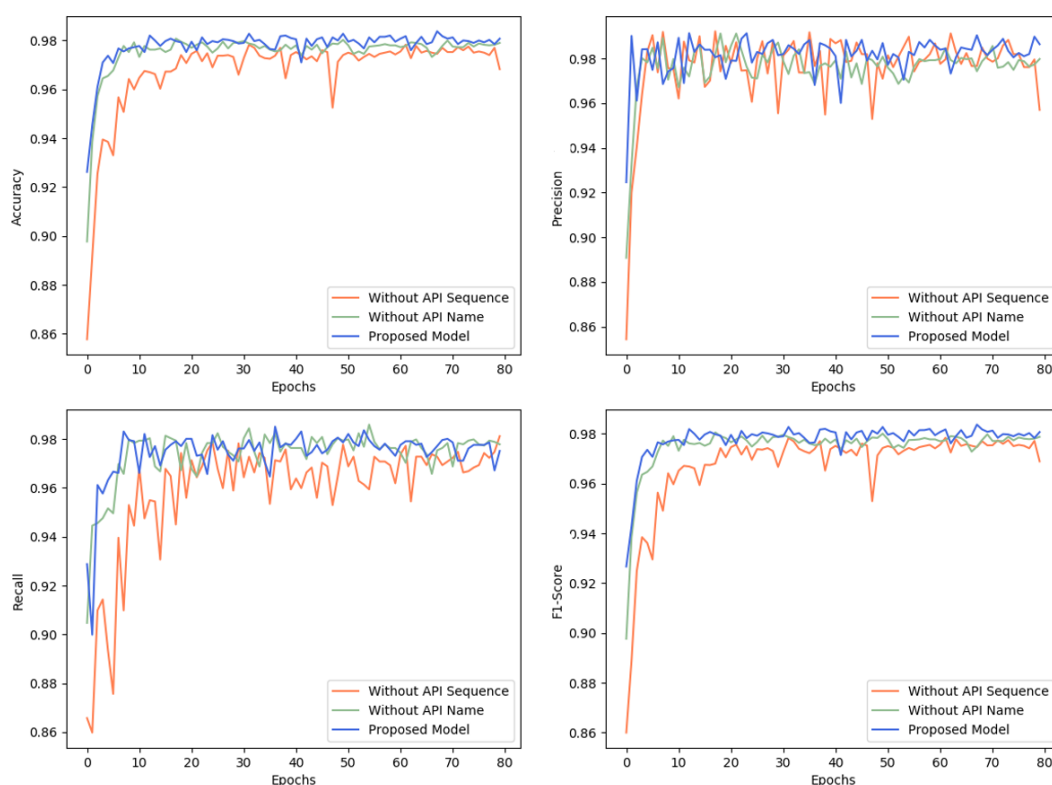


图 3.9 不同特征处理模型的准确率、精确度、召回率以及 F1-Score

（4）应用不同卷积层的性能比较

本实验旨在研究不同卷积层数量下恶意代码检测模型效果的变化。为此，设计了三组实验，具体实验配置包括两个卷积层，滤波器大小分别为 3、5；三个卷积层，滤波器大小分别为 3、4、5；三个卷积层，滤波器大小分别为 3、5、7；以及四个卷积层，滤波器大小分别为 3、4、5、6。

图 3.10 显示了卷积层在不同配置下的性能表现。观察到，卷积层数量为 2 时，模型的收敛速度相对较慢，且最终性能最差。而当卷积层数量增加到 3 时，模型的性能显著提高，超过了卷积层数量为 2 的情况。在卷积层数量为 3 的情况下，进一步研究了不同滤波器大小的性能差异，发现滤波器大小为 3、4、5 时的模型性能要优于滤波器大小为 3、5、7 的情况。此外，当卷积层数量增加到 4 时，并未带来性能的进一步改善。因此，本方法选择了三层卷积层结构，应用的滤波器大小分别为 3、4、5。

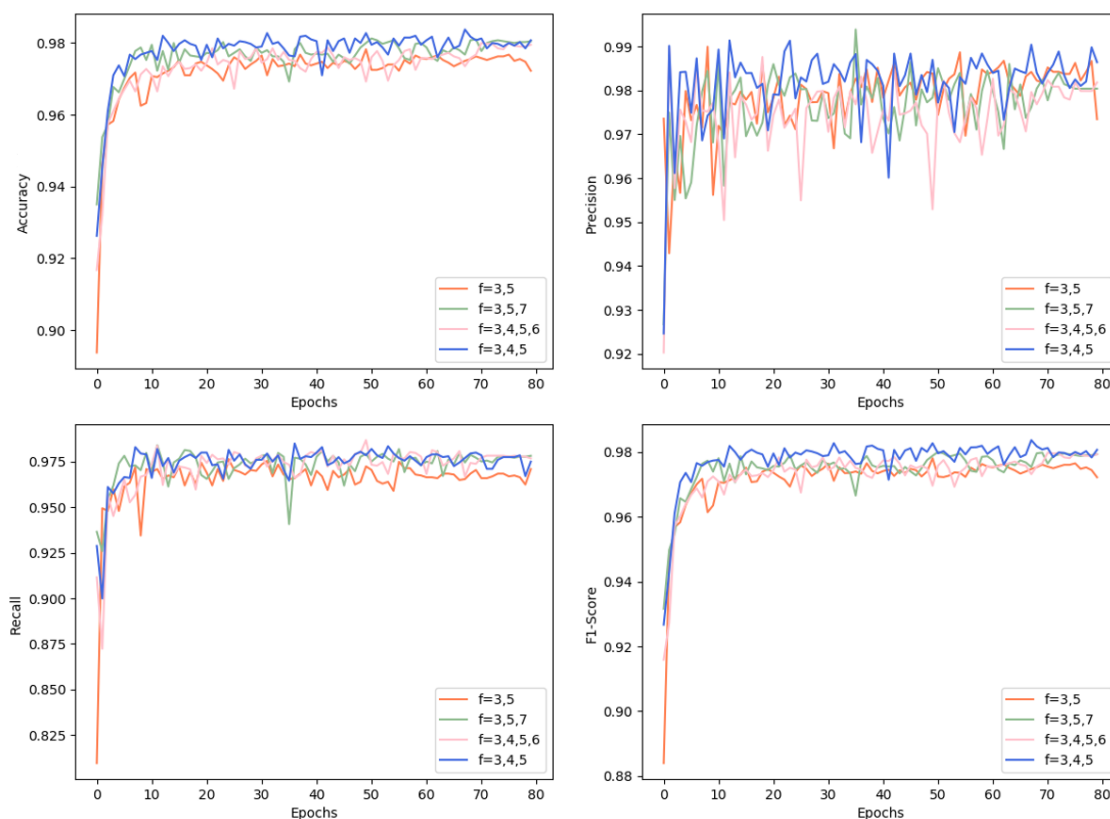


图 3.10 应用不同卷积层的准确率、精确度、召回率以及 F1-Score

（5）GRU 模型对检测性能的影响

为分析实验设置的 BiGRU 层对检测模型性能的影响，设计了三种模型进行分析比较：模型不包含 BiGRU、只有一个单向 GRU，以及有两个 GRU 堆叠构成的 BiGRU。

图 3.11 显示了不同数量的 GRU 对模型性能的影响。观察到，BiGRU 模型在准确率、召回率以及 F1 分数上均明显优于其他两个模型，收敛速度更快，随着训练周期的增加，准确率逐步提升稳定。验证了 BiGRU 通过引入双向学习，能够更全面地捕捉 API 调用序列中的上下文关系，从而提高了模型对输入数据的理解和表征能力。值得注意的是，随着 GRU 数量的增加，模型的收敛速度逐渐加快，性能也在不断提高。反映了通过增加 GRU 的数量，模型能够更灵活地学习 API 调用之间的上下文依赖关系，从而在恶意代码检测任务中实现更为精准的预测。

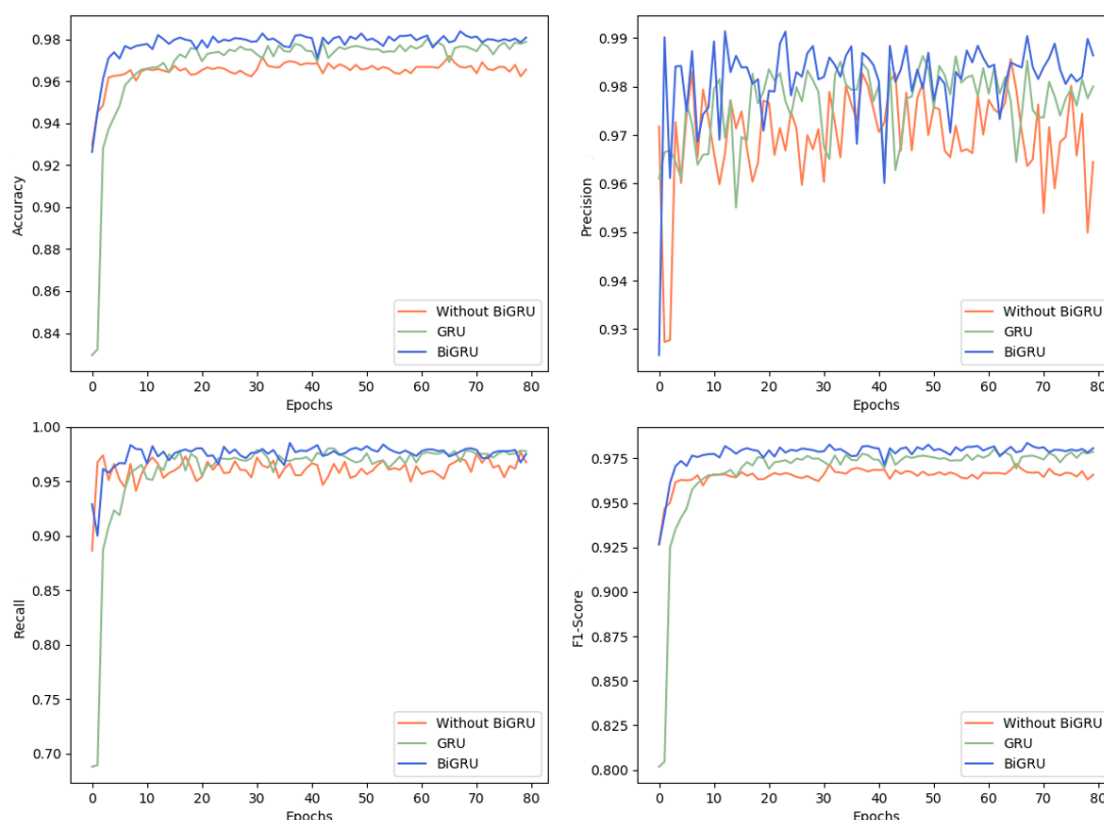


图 3.11 应用不同数量 GRU 的准确率、精确度、召回率以及 F1-Score

(6) 与其他先进方法比较

为了研究所提模型的性能改进，本实验将所提模型与其他研究模型进行了比较。在本节中，本实验复现了下述相关文献中的方法，并使用相同数据集进行实验。Kolosnjaji 等人^[67]构建了一个基于卷积和递归网络层的神经网络。使用 One-Hot 编码来为数据集中的每个 API 调用构造二进制向量，并使用包含两个卷积层和一个递归神经网络的混合神经网络来进行恶意代码分类。Liu 等人^[27]提出了一种基于 BLSTM 的恶意代码检测方法。删除了连续出现两次以上的相同 API，并将过滤后的 API 调用按照调用时间进行排序，然后使用 Word2Vec 进行词嵌入，

这里他们将向量长度设置为 50。最后使用 BLSTM 神经网络实现恶意代码检测。Catak 等人^[68]构建了基于深度学习方法的恶意代码检测模型，通过使用嵌入层和两层 LSTM 对收集的 API 调用序列进行检测。Li 等人^[64]提出了一种基于 API 序列内在特征的恶意代码检测框架，设计了一个由 3 个模块组成的内在特征编码器来表示和组合 API 序列的内在特征。基于特征编码器，实现了一个深度神经网络模型来检测软件是否恶意。

最终结果如表 3.4 所示，与所有其他先进检测模型相比，本章提出的模型在所有指标中得分最高，优于现有的检测技术。与基于传统方法的模型相比，例如 Kolosnjaji 等人和 Catak 等人的研究，本章所提方法学习更多的特征并且模型性能更高效，这表明通过词嵌入模型获取 API 的语义和结构信息对于提高性能具有重要影响。Liu 等人仅使用语义特征，Li 等人仅使用结构特征，两者都获得了良好的结果，但都不是性能最好的方法，这表明结合结构语义信息和 API 统计特征能够更充分地学习 API 的特征，从而提高恶意代码检测的效果。

表 3.4 与其他先进方法比较

	Accuracy	Precision	Recall	F1-Score	结构	语义
Kolosnjaji et al. ^[67]	0.9660	0.9569	0.9775	0.9671	×	×
Liu et al. ^[27]	0.9668	0.9763	0.9575	0.9668	×	✓
Catak et al. ^[68]	0.9685	0.9720	0.9642	0.9681	×	×
Li et al. ^[64]	0.9738	0.9774	0.9712	0.9743	✓	×
Proposed Model	0.9828	0.9870	0.9789	0.9827	✓	✓

3.8 本章小结

本章提出了一种基于 API 调用序列的恶意代码检测的新方法，该方法结合了词嵌入和深度学习技术，从行为序列和名称结构两个维度全面分析恶意代码的动态 API 调用。首先，运用词嵌入技术将 API 调用序列进行向量化表示，通过 Skip-Gram 模型进行训练，以获取 API 的上下文语义信息。然后，通过分析调用名称结构的语义信息和统计特征，对 API 调用名称进行编码。最后，将 API 序列特征向量和 API 名称特征向量输入到 CNN-BiGRU 网络以训练恶意代码检测器。多组实验表明，本章检测方法优于所提基线，可以有效实现恶意代码检测。

第4章 基于 API 异构参数增强的恶意代码检测方法

4.1 引言

在不断演变和进化的网络威胁中，恶意代码已逐渐成为黑客在全球范围内针对个人用户以及各类规模企业发动多样化网络攻击的首选自动化工具。根据统计数据显示，新出现的恶意代码攻击事件中，有超过 95% 的案例均以 Windows 操作系统设备作为主要目标，保护 Windows 设备免受恶意代码侵害已成为当前网络安全防护工作的重点之一。传统的恶意代码检测方法往依赖于特征匹配、签名扫描等技术，在面对恶意代码的指数级增长态势以及日益复杂的混淆与逃避策略时，逐渐暴露出其局限性。因此，结合深度学习等先进算法的动态行为分析技术逐渐成为恶意代码检测领域的重要研究方向，以提升恶意代码检测的有效性和准确性。

然而，当前基于深度学习的动态恶意代码检测技术仍面临一些挑战。一方面，现有的大多数基于动态分析的恶意代码检测方法提取的特征单一，往往仅依赖于对 API 调用名称的分析，而对传递给 API 的参数信息的研究有所遗漏。虽然现有部分研究使用哈希编码等技术对参数特征进行统一编码处理，但此类通用化处理方式无法深入研究异构类型参数的内在含义，同时编码过程也会带来大量的时间和计算开销。另一方面，当前深度神经网络模型普遍面临复杂性高和参数量大等问题，在训练过程中消耗大量的计算资源和存储空间。此外，尽管深度神经网络在特定任务上取得了显著的性能提升，但它们往往面临着泛化能力不足的问题，这限制了其在实际应用中的广泛性和可靠性。因此，需要不断强化安全策略、更新检测手段，从而为网络安全防护提供更为全面且精准的技术支持，以有效抵御日益增长且不断演化的恶意代码攻击。

为了解决上述问题，本章提出了一种基于 API 异构参数增强的恶意代码检测方法。该方法通过对恶意代码动态运行报告进行深度挖掘，不仅关注 API 调用序列的提取与分析，还对 API 运行时的参数特征进行了细粒度的语义信息分析，从而显著提升了恶意代码检测的全面性和准确性。同时，设计和应用了参数需求量小的神经网络模型，以较低的复杂度学习 API 序列远程依赖关系和 API 参数的深层次结构特征，在增强检测模型的表达能力同时提升了模型的泛化性能。

4.2 方法框架

本节将详细介绍基于 API 异构参数增强的恶意代码检测方法，总体框架结构如图 4.1 所示。

该模型主要包括三部分：数据预处理、构建 API 序列和 API 参数模型以及特征融合。首先，在动态运行报告中提取所需 API 和相关参数信息；然后，引入 Doc2Vec 嵌入模型并设计使用了双向时域卷积网络学习 API 调用序列；同时，根据异构类型参数的结构功能对 API 参数进行切片划分，并学习划分后参数特征的语义信息，利用轻量化的神经网络模型 EfficientNet 学习 API 参数的特征；最后，将序列模型和参数模型的输出特征融合，引入全连接层和 AdamW 优化器，实现最终的检测。

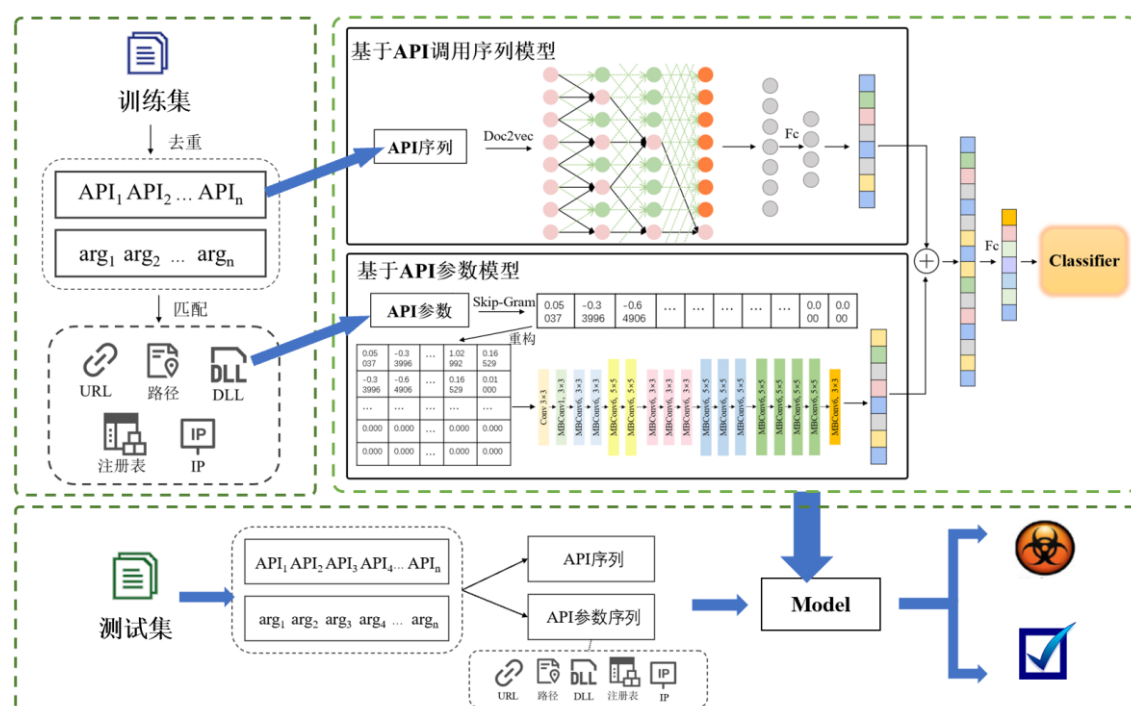


图 4.1 基于 API 异构参数增强的恶意代码检测的总体架构图

4.3 数据预处理

本研究基于恶意代码的动态运行报告，对所需的 API 调用序列和 API 运行参数信息进行提取，具体过程如算法 4-1 所示。首先，在特征提取的过程中判断所提取 API 调用是否为冗余序列，仅保留非连续重复的调用并提取其运行参数。然后，针对筛选后的 API 调用参数信息，本节进行了更为深入的处理。考虑到 API 调用运行时参数类型的多样性，为了更精准地捕获关键信息，本节引入了正则表达式的匹配机制。具体而言，重点关注了五种参数类型，包括 URL、路径

(path)、动态链接库 (DLL)、注册表和 IP 地址。通过关注恶意代码对操作系统的网络连接、文件管理、注册表访问等行为,有助于深入了解该代码的行为目的。

算法 4-1 提取 API 调用序列和运行时参数数据

算法 4-1 提取 API 调用序列和运行时参数数据

输入:

D_j : 动态分析报告

输出:

API_calls, parameters

```
1: API_calls = [] //存储 API 序列
2: parameters = [] //存储 API 参数信息
3: previous_API = None // 用于比较前一个 API 调用是否与当前相同
4: xmldoc = minidom.parse( $D_j$ ) // 解析 XML 格式的报告
5: actionlist = xmldoc.getElementsByTagName('action')
6: for s in actionlist:
7:     api = s.attributes['api_name'].value //获取 API 调用
8:     // 若当前 API 不等于前一个 API 或者前一个 API 为空, 则保留
9:     if api != previous_API or previous_API is None:
10:         API_calls.append(api)
11:         parameters_for_this_api = []
12:         params=s.childNodes[1]
13:         for i in range(len(params.childNodes)):
14:             //获取参数值
15:             parameter_value = params.childNodes[i].attributes['value'].value
16:             for type in [URL, path, DLL, registry, IP_address]:
17:                 if matches_type(parameter_value):
18:                     parameters_for_this_api.append(parameter_value)
19:             end for
20:         parameters.append(parameters_for_this_api)
21: end for
```

4.4 API 序列模型

4.4.1 API 序列嵌入

完成特征提取后，进一步对 API 序列进行词向量训练，以反映其语义信息。然而，一个潜在的问题是不同代码的 API 调用数量存在差异，因此在向量化过程中易导致生成的序列特征向量维度差距较大问题。为了有效地对 API 序列进行语义建模，本研究使用 Doc2vec 模型进行训练。Doc2vec^[69]由 Mikolov 等人在 2014 年提出，作为 Word2vec 的扩展，它在 Word2vec 基础上添加了一个唯一的段落向量 Document ID，专门用于生成句子、段落或文档的嵌入向量^[70]。

本研究基于 Gensim 库来构建并训练 Doc2vec 模型，并选用分布式存储（PV-DM）模型对提取的 API 调用序列进行训练，如图 4.2 所示。

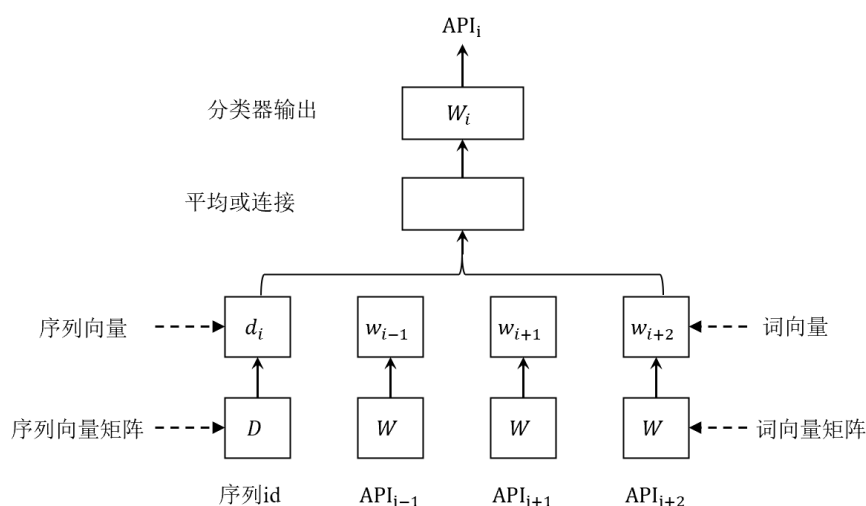


图 4.2 PV-DM 模型图

在该模型中，每个样本的 API 序列 S_i 都被映射为一个独立的序列向量 d_i ，记为矩阵 D 中的一行。同时，序列 S_i 中的每一个调用 API_i 也被映射为一个独立的向量 w_i ，作为矩阵 W 中的一列。对于给定的序列调用，该模型的任务是得到词向量目标函数的最大化平均概率，其目标函数的数学表达式由公式（4.1）所示。

$$\frac{1}{T} \sum_{i=k}^{T-k} \log P(w_i | d_i, w_{i-k}, \dots, w_{i+k}) \quad (4.1)$$

其中，条件概率函数 $P(w_i | d_i, w_{i-k}, \dots, w_{i+k})$ 的计算过程如下所示：

$$P(w_i | d_i, w_{i-k}, \dots, w_{i+k}) = \frac{e^{y\omega_i}}{\sum_j e^{y_j}} \quad (4.2)$$

其中， T 为参与训练的调用总数， k 为训练窗口的大小， d_i 为当前窗口中词的上

下文所在序列的序列向量。

在上述条件概率表达式中, y_i 表示第 i 个输出的 API 未归一化的 $\log$ 概率值, 计算公式如下:

$$y_i = b + Uh(d_i, w_{i-k}, \dots, w_{i+k}, W, D) \quad (4.3)$$

其中 U, b 为 softmax 函数的参数, h 为从词向量矩阵 W 和序列向量矩阵 D 抽取的元素通过特定的聚合操作 (如求均值或连接)。

本研究设置训练的序列向量维度为 784, 上下文窗口大小为 3, 最小计数设为 3。通过对上节中所提取的 API 调用序列进行训练, 每个序列都被表示为一个维度为 784 向量。训练完成后, 将生成的向量模型以及序列对应的向量保存, 以备后续直接加载和使用。该序列向量将作为后续的双向时域卷积网络的初始嵌入向量进行训练。

4.4.2 双向时域卷积网络

尽管 RNN 及其变体被广泛用于顺序输入, 但其训练成本巨大, 存在能量消耗问题和长时间消耗问题^[71]。为了有效应对这一问题, 本节设计了一个双向时域卷积网络 (BiTCN)。该网络利用了 TCN 的深层网络结构和并行计算能力, 能够以较低的计算复杂度高效地处理顺序输入, 且有效地捕捉序列中远距离上下文依赖。

传统的 TCN 是单项结构, 其中的因果卷积具有严格的时间约束, 使神经元在 t 时刻只能看到在该时刻之前的信息, 而不能看到之后的任何信息, 预测仅依靠历史信息。由于 API 之间的上下文关系是双向依赖的, 因而单向的 TCN 模型在学习整体语义方面存在一定局限性, 不利于序列特征的充分表达。本节在传统 TCN 的基础上将 TCN 扩展到双向网络, 即 BiTCN 模型。该网络在处理输入序列时具有正向和反向两个方向, 使模型能够捕获过去和未来时间步长之间的相互依赖关系, 如图 4.3 所示。双向 TCN 的最终输出通过连接前向和后向传递的输出获得。这种双向结构的引入增强了模型对序列数据的建模能力, 有助于更全面地理解序列中的复杂关系。

前向特征 h^{fwd} 和后向特征 h^{bwd} 的计算过程可表示为:

$$h^{fwd} = \delta(W_1 y^{fwd} + b_1) \quad (4.4)$$

$$h^{bwd} = \delta(W_2 y^{bwd} + b_2) \quad (4.5)$$

其中 W 和 b 分别为权重矩阵和偏置项, 它们均为可训练的参数。 δ 为激活函数, 该模型使用的是 ReLU 激活函数。

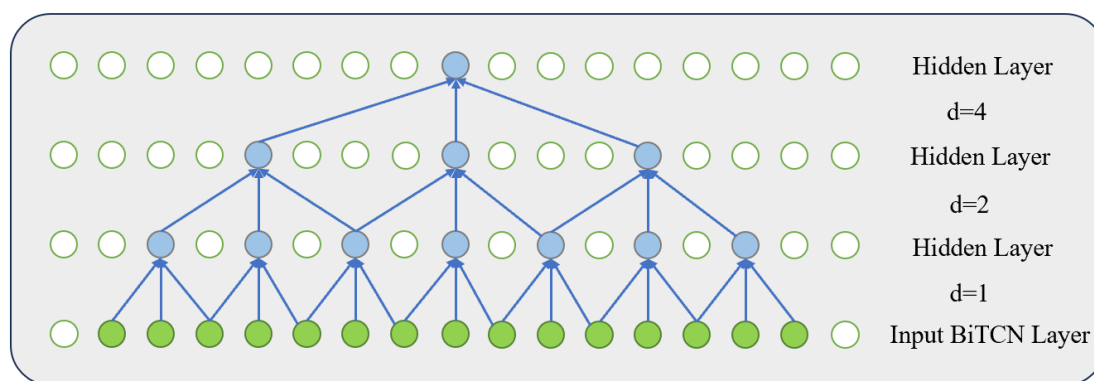


图 4.3 BiTCN 示意图

双向 TCN 的最终输出可表示为:

$$h = [h^{fwd}, h^{bwd}] \quad (4.6)$$

4.5 API 参数模型

4.5.1 API 参数嵌入

鉴于 API 运行时参数不同位置的标记具有不同的功能特征，为了更精确地学习参数特征，本节对 URL、文件路径、DLL、注册表项、IP 地址这 5 种类型的参数进行了切片分解。参数的具体切片规则如表 4.1 所示。

表 4.1 参数切片规则

参数类型	切片数量
URL	4
文件路径	3
DLL	4
注册表项	3
IP地址	5

对于类型为 URL 的参数，本节根据“/”的位置将 URL 分解成四个关键部分，分别是协议部分、域名部分、路径部分和标识部分，如图 4.4 所示。其中，协议部分明确了数据传输所遵循的规则；域名部分则用于明确目标主机的地址标识，通常是一个可读的名称；路径部分描述资源在服务器上的位置，形成了层级结构；标识部分用于标识资源的具体标识符，包含了资源名称或查询参数等信息。

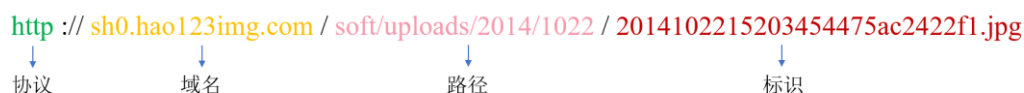


图 4.4 URL 分解示例

对于动态链接库类型的参数，利用正则表达式将其路径分解为四个片段。具体来说，首先基于首个斜杠“/”来分隔出驱动器部分，这部分表示了动态链接库实际存储的硬盘或分区；接着，通过匹配第二个斜杠，进一步划分出系统核心目录或用户数据目录，这有助于理解动态链接库在系统结构中的位置；最后，利用路径中最后一个斜杠“/”作为界定点，分隔出路径目录和文件名。类似地，对于文件路径，通过识别第一个和最后一个斜杠来将其划分为三个关键区域：驱动器、路径目录及文件名。这种划分方式有助于本研究在分析过程中明确区分文件存储位置以及文件名，从而确保对文件路径的理解和操作更加精确和高效。

注册表路径的划分也借鉴了这一思路，利用正则表达式匹配首个和第二个斜杠“/”，将注册表项划分为注册表根键、主分支和子分支三部分。其中，注册表根键是整个注册表层次结构的起始点，而主分支和子分支则进一步细化了层次结构。这种划分方式不仅简化了注册表项深层次结构的理解难度，还能灵活适应不同配置下的注册表项的分解操作，降低了分析复杂性。

在处理 IP 地址的切片时，观察到部分 IP 信息除了包含标准的 IPv4 地址之外，还额外附带了端口号信息，通常采用冒号“:”作为界定符号。因此，本研究采用点“.”和冒号“:”这两种符号对 IP 地址进行匹配和切片处理。具体而言，本节首先通过冒号“:”将 IP 地址与其对应的端口号进行有效分离。随后，利用点“.”符号将 IP 地址划分为四个部分，每个部分都是一个介于 0 至 255 之间的整数，从而实现对 IP 地址及端口信息的精确解析与定位。

分解完成后，将生成的 API 参数切片序列输入到 Word2Vec 模型中进行训练，本研究使用 Skip-Gram 模型来训练参数切片特征的矢量化表示，以捕获参数片段之间的语法和语义关系，设定参数切片特征向量的维度为 20。完成训练后，将每个应用程序的所有参数特征向量进行合并。观察到，每个应用程序的特征向量维度大都在 50,000 以内，为了尽可能地包含更多的特征信息，本方法进一步调整参数，将参数向量的维度设定为 50,176。对于维度超过该数值的特征向量，对其进行截断，而对于维度不足的特征向量则进行补零处理。

4.5.2 轻量化神经网络模型

随着神经网络模型的不断发展和加深，模型的规模和计算复杂度也在逐步增加。然而，这些规模更大的模型对计算资源的需求也随之增加，且更容易陷入过

拟合问题。因此，为了解决上述问题，本节引入轻量化模型 EfficientNet。EfficientNet 模型^[72]由 Google Brain 团队在 2019 年提出，该网络采用 MBConv（Mobile Inverted Bottleneck Convolution）模块作为基础模块，每个模块包含多个卷积层和标准化层，如图 4.5 所示。此外，在模块之间，还设计了降采样层以减小特征图的尺寸，进一步提升模型的效率。通过采用 EfficientNet，检测模型能够学习单个参数或参数组合在恶意代码检测中的多样性贡献，从而在实现高效检测的同时，有效降低过拟合的风险。

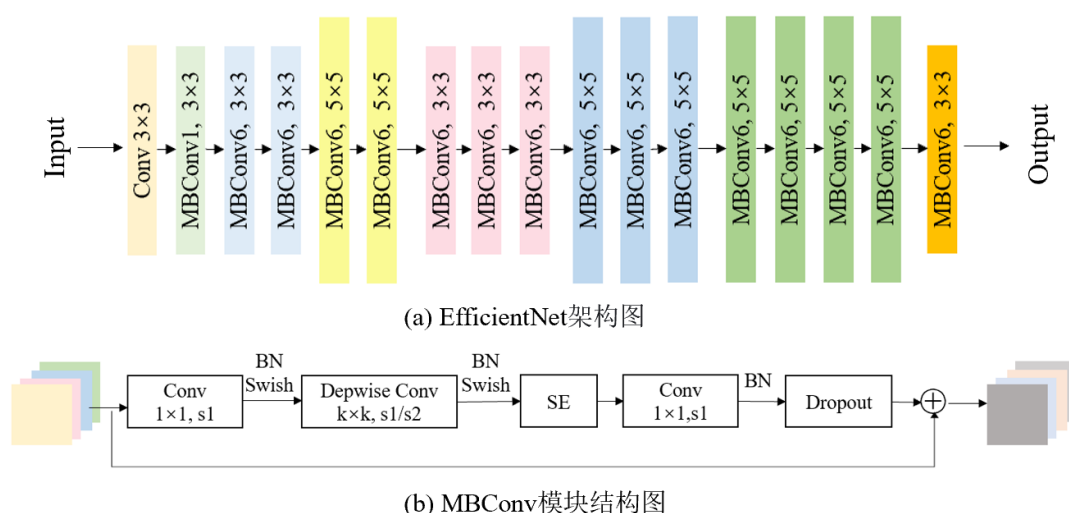


图 4.5 EfficientNet 网络架构图

为了使参数特征匹配模型的输入，本节采用特征重构技术，将原始特征矩阵从 $(m, n, 1)$ 的形式转化为 $(m, \sqrt{n}, \sqrt{n})$ ，其中 m 代表样本数量， n 表示每个样本特征向量的维度。通常，可以利用线性变换或者其他方式通过降维或升维的手段确保 $\sqrt{n}$ 为整数。在本研究中，每个应用程序的参数特征向量为 50,176 维，因此这里直接将参数特征转换成大小为 224×224 的特征矩阵，将其作为参数模型的输入数据，并通过 EfficientNet 网络模型进行训练。为了进一步提升检测的性能，本节在模型的最后接入了多层全连接层，将输出的特征维度从较高维度逐步将至较低的 64 维。通过降低特征维度，减少后续模型的计算复杂度，提高训练效率。

4.6 特征融合

通过使用 cat 函数，将序列模型和参数模型的输出特征融合在一起，融合后的特征维数为 128。然后，将拼接后的特征传入三层全连接网络进一步进行深化处理，各层神经元数依次为 128、64、32。最后，通过输出层进行分类，应用 Sigmoid 激活函数得到最终的检测结果。通过融合两个模型的输出，本方法能够

充分利用不同模型的优势，在一定程度上弥补单一模型的不足，从而更好地适应各种恶意代码的演化，提高检测模型的准确性与鲁棒性。

在模型训练阶段，选用二分类交叉熵损失函数对融合模型进行训练，训练过程中通过反向传播和梯度下降等优化算法来调整网络参数，以优化整体的二分类交叉熵损失，从而实现损失的最小化。损失函数计算公式如下所示：

$$L(y, \hat{y}) = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})] \quad (4.7)$$

其中， y 为样本的实际标签； $\hat{y}$ 是模型对样本的预测输出，即通过激活函数得到的概率值。

同时，利用 AdamW 优化算法来对网络模型的参数进行高效且稳健的更新。AdamW 优化器在 Adam 的基础上加入了权重衰减，以此提高优化器的泛化能力，参数更新公式表示为：

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t \quad (4.8)$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2 \quad (4.9)$$

$$\hat{m}_t = \frac{m_t}{(1 - \beta_1^t)} \quad (4.10)$$

$$\hat{v}_t = \frac{v_t}{(1 - \beta_2^t)} \quad (4.11)$$

$$\theta_t = \theta_{t-1} - \eta \left[\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda \theta_{t-1} \right] \quad (4.12)$$

其中 g_t 为本次迭代的梯度； m_t 为 g_t 各分量的均值， $\hat{m}_t$ 、 $\hat{v}_t$ 为校正后的值； β_1 、 β_2 为超参数，分别为 0.9 和 0.999； v_t 为 g_t 各分量的平方的均值； $1 - \beta_1^t$ 和 $1 - \beta_2^t$ 为修正系数； ϵ 为一个极小的常量，通常为 1×10^{-6} ； θ_t 为模型权重、 θ_{t-1} 为上一步中的模型权重； $\eta \in \mathbb{R}$ 为学习率； $\lambda \in \mathbb{R}$ 为正则化系数。

4.7 实验与分析

4.7.1 实验环境与数据

本研究使用了由第三方在 GitHub^[65] 平台上公开分享的大规模数据集，该数据集由 Windows PE 文件在安全可控的沙箱环境中执行而产生的行为记录构成。如图 3.6 所示的行为记录片段中，展示了相关的行为数据，其中“exInfo value”这一字段特指了 API 在执行过程中所使用的关键参数值。本研究从该数据集中抽取了 20,000 个 PE 文件的执行记录，其中 10,000 个为良性样本，其余为恶意样本。这些样本共涉及 94 个的 API。观察到文件的 API 序列长度分布区间广泛，为了有效地控制分析的复杂性和资源消耗，将所有 API 序列的最大长度限制为 1000。

表 4.2 实验参数

实验参数	取值	实验参数	取值
Doc2Vec 模型窗口大小	3	Skip-Gram 模型窗口大小	3
Doc2Vec 模型向量大小	784	Skip-Gram 模型向量大小	20
Doc2Vec 模型负采样数	3	Skip-Gram 模型负采样数	5
Doc2Vec 模型最小计数	3	Skip-Gram 模型最小计数	3

所提模型使用 python 语言编写实验程序，并在 NVIDIA RTX 3080 GPU 上进行训练。本章实验中 Doc2Vec 模型和 Skip-Gram 模型的训练参数设置如表 4.2 所示。此外，BiTCN 模型的卷积层数设置为 4，卷积核大小为 7，扩张因子数为 2^n 即 [1, 2, 4, 8]。在训练过程中，检测模型的学习速率设定为 10^{-3} ，Dropout 率为 0.2，实验 Batch Size 为 64，训练周期为 80，以确保模型在足够的数据迭代次数中能够学习到足够的特征和模式，从而提高模型的性能和泛化能力。

4.7.2 评价方法与指标

本实验同样使用准确度（Accuracy）、精确度（Precision）、召回率（Recall）和 F1-Score 等评估指标来评估所提模型的性能，计算公式如上述 3.7.2 所示。

4.7.3 实验结果分析

（1）不同数据集划分比率下对检测准确率的影响

本实验的主要目标是探究不同比例的训练集与测试集对恶意代码检测准确率的影响。为此，本节采用四种不同的划分标准进行对比分析，具体为将数据集按照 9:1、8:2、7:3 和 6:4 的比例进行训练集与测试集的划分，以全面评估不同比例下的检测性能差异。图 4.6 描述了训练集的准确率和损失值随训练周期的变化情况。图中横轴代表训练周期数，共设置了 80 个周期；纵轴则分别表示训练集的准确率及损失值。观察发现，随着周期的增加，训练集的准确率呈现出不断上升的趋势，且损失值逐渐下降。当训练到一定程度时，准确率和损失值的变化逐渐趋于平稳，这表明模型已经趋于稳定。测试集的准确率和 F1-Score 随训练周期的变化情况如图 4.7 所示。观察到，在训练集和测试集比例由 6:4 逐渐调整为 7:3 至 8:2 时，模型的准确率和 F1-Score 均不断上升，模型的性能有所提升。这反映了在适当增加训练集比重时，模型能够从更为丰富多样的训练样本中学习更多的信息和特征，从而增强了对恶意代码样本的识别能力和泛化性能。然而，在当训练集和测试集比例设置为 9:1 时，模型的性能并未出现相应的提升。这可能是因为在 9:1 的比例下，模型过度学习训练数据中的细节和噪声，因而在新数据上表现不佳，从而造成了过拟合。因此在实际应用中，合理分配训练集与测试集比例对于保障模型的有效性和泛化能力十分重要。

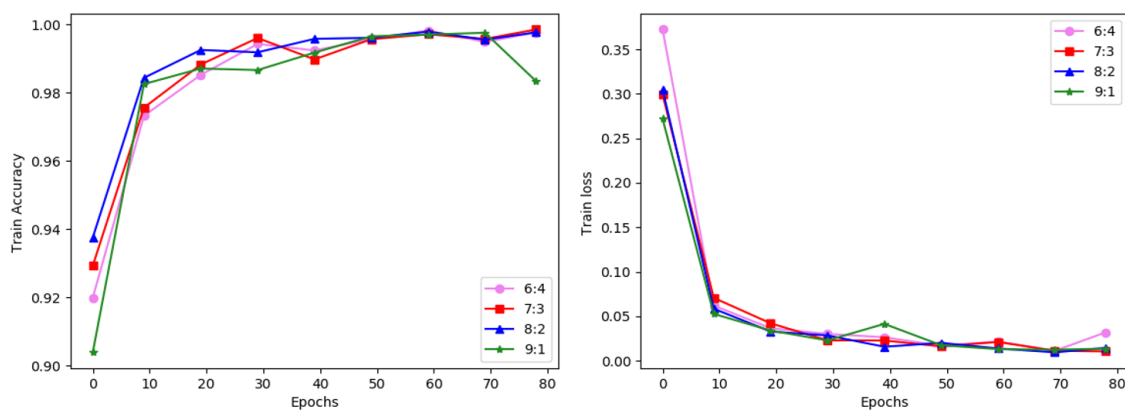


图 4.6 训练集的准确率和损失值随训练周期的变化趋势

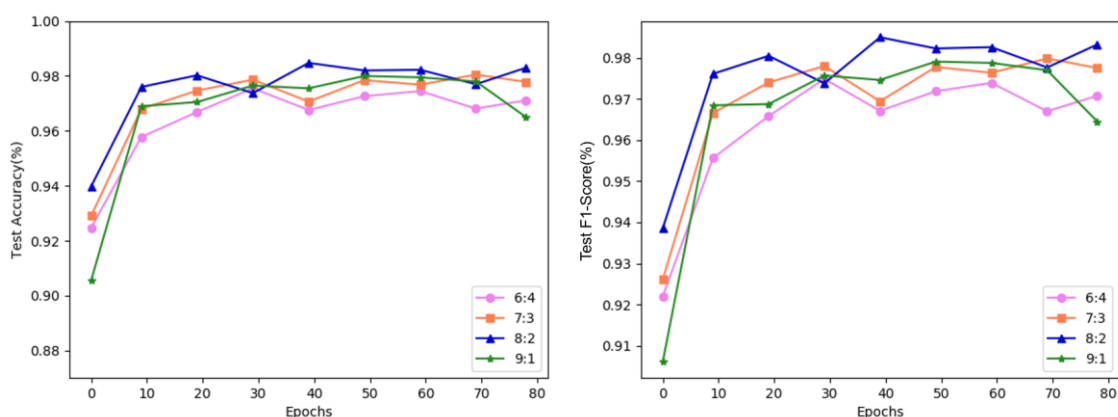


图 4.7 测试集的准确率和 F1-Score 随训练周期的变化趋势

(2) 不同特征处理模型的性能比较

为了准确评估 API 序列模型和 API 参数模型对恶意代码检测效果的贡献,本节分别将序列特征和参数特征从检测模型中移除,然后对剩余模型的性能进行独立评估,实验结果如图 4.8 所示。通过对比分析不同特征处理模型在测试集上的准确率、精确度、召回率以及 F1-Score 等指标,发现尽管使用单一特征的两个模型都能在一定程度上识别出恶意代码,但它们的检测效果并不突出,检测准确率均为 94%左右。相比于本文所提模型,仅使用 API 序列的模型性能降幅较大,准确率降低 4%,F1-Score 降低 5%。此外,API 参数模型在总体性能上较 API 序列模型更为出色,这表现了学习运行参数所反映出的行为信息的重要性。然而,值得注意的是,在对测试集的评估过程中,API 参数模型的各项评估指标变化幅度相对较大,表明该模型在泛化能力上存在不足。综合考虑各项性能指标发现,融合了 API 序列特征和 API 参数特征的模型表现最为出色,收敛速度更快。当准确率趋于稳定时,该模型主要在 98.0%到 98.5%之间轻微波动。图 4.9 比较了三种模型的 ROC 曲线,可以看出同时本章提出的模型表现出了最为优越

的性能，该模型的 AUC 值最高，达到了 0.9974。实验结果反映了通过结合 API 序列特征和 API 运行时参数特征，有助于模型更全面地捕捉恶意代码的行为特征，模型能够更有效地解析并识别出恶意代码的行为轨迹，从而提高检测的准确性和可靠性。

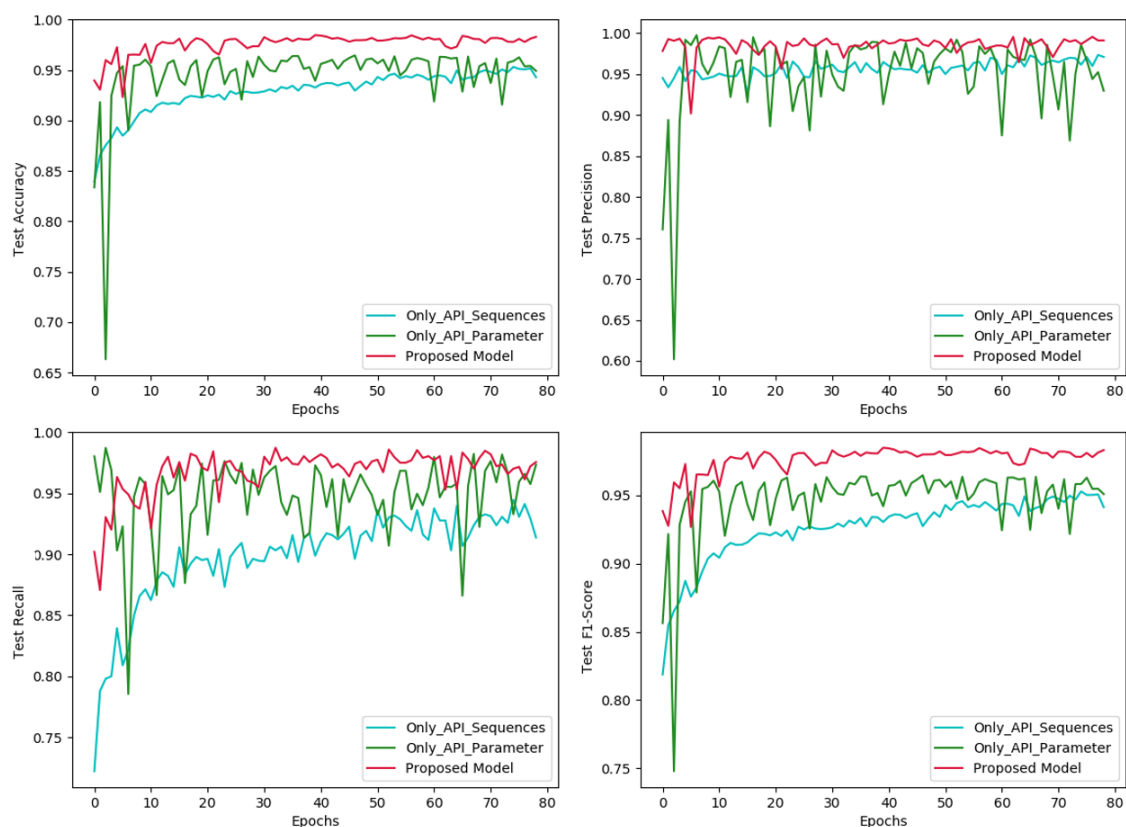


图 4.8 不同特征处理模型的准确率、精确度、召回率以及 F1-Score

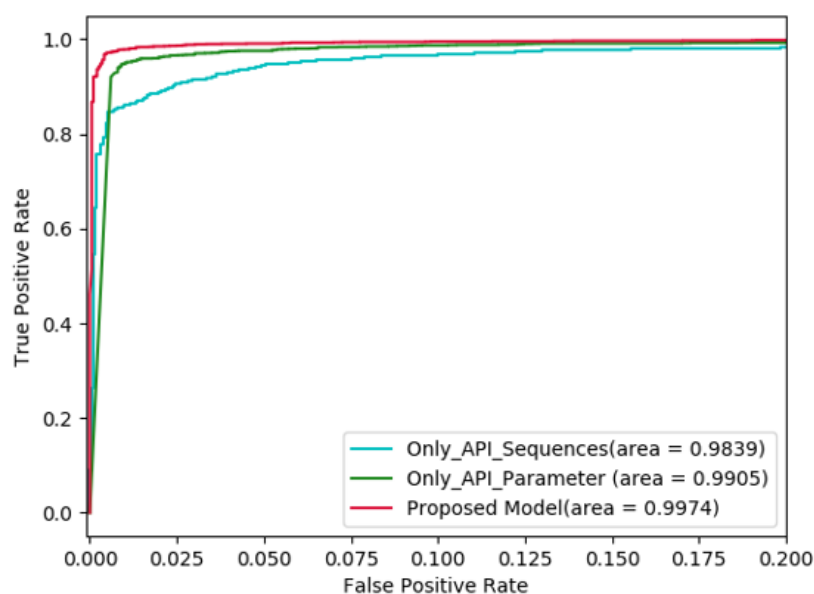


图 4.9 不同特征处理模型的 ROC 曲线

（3）双向时域卷积网络对检测性能的影响

在提取 API 调用序列语义特征后，本节使用四种不同的方法，分别为 LSTM、TCN、BiLSTM、BiGRU 以及 BiTCN，以全面评估和验证它们在恶意代码检测任务中的性能表现。如图 4.10 所示，其中横坐标代表测试集的准确率、精确度、召回率和 F1-Score，纵坐标为评价指标的数值。图中观察到 BiTCN 模型相较于单向的 TCN 模型展现出更为出色的性能，其各项指标均达到了 1%的提升。这验证了在处理数据时，双向模型能够更全面地捕捉和理解序列的上下文信息，尤其是当结合了时域卷积网络的捕获长距离依赖能力后，能更充分地挖掘序列语义的特征，从而提高了模型的表现。图 4.11 显示了不同模型的 ROC 曲线图。在双向模型的比较中，BiTCN 的整体性能最佳，其 AUC 值高达 0.9974。其次是 BiGRU，其 AUC 值为 0.9939，而 BiLSTM 模型的 AUC 值则为 0.99。这可能因为 BiLSTM 与 BiGRU 模型具有更多的参数，容易在训练过程中出现过拟合，而在 BiTCN 基于因果卷积和空洞卷积，进一步丰富了模型的表达能力，但又不会过度增加模型的复杂度。值得注意的是，BiLSTM 虽然拥有良好的序列建模能力，但效果不如 LSTM 理想，准确率仅为 96.7%。这可能是在训练过程中，模型参数量相对较大，容易因为模型复杂度过高而过拟合训练数据。

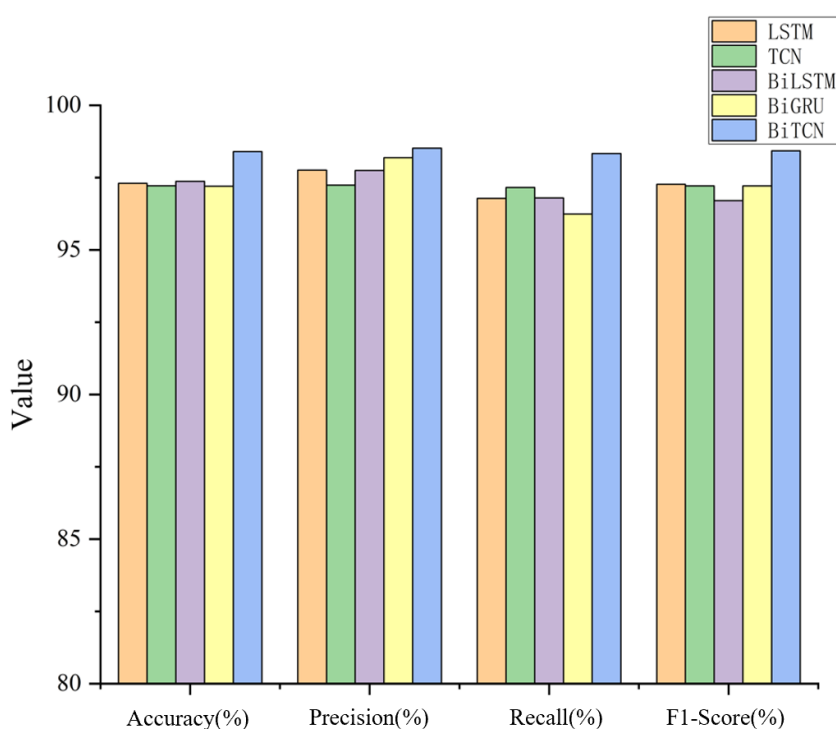


图 4.10 不同模型的准确率、精确度、召回率和 F1-Score

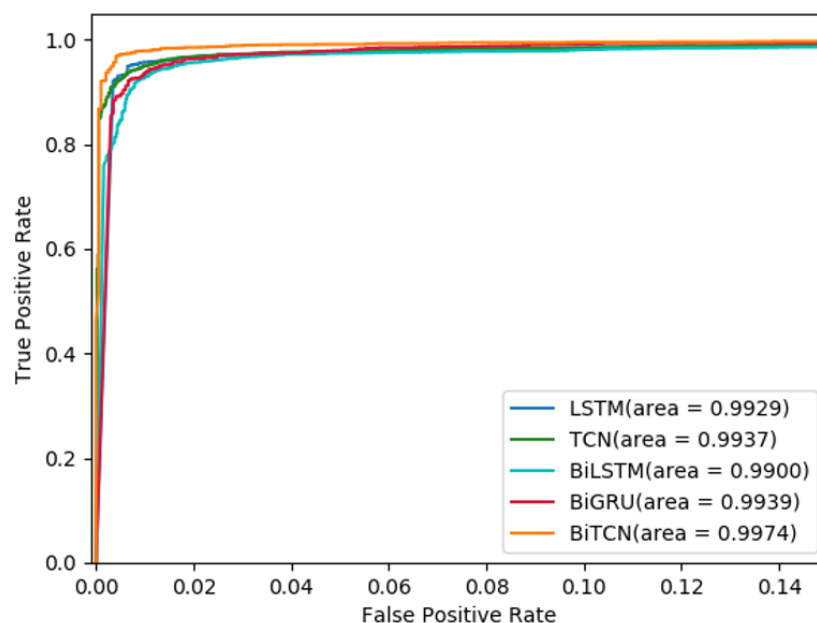


图 4.11 不同模型的 ROC 曲线

（4）不同词嵌入模型的性能比较

本实验针对恶意代码参数特征使用不同词嵌入模型与向量大小时的检测效果进行对比分析，实验结果如表 4.3 所示。实验结果表明，使用 Skip-Gram 的方法在各项评估指标下都取得了很好的效果，整体优于 CBOW 算法。值得注意的是，当嵌入向量设为 20 时，所有的词嵌入模型都达到了最高的准确率，均在 97% 以上。此外，观察到在嵌入向量维度由 10 增至 20 的过程中，模型的检测性能呈现显著提升，然而，当继续将维度由 20 提升至 30 时，模型的性能并未随之进一步增强。可能是由于过高的维度导致模型学习到噪声而非有用信息，影响模型的优化过程与最终性能。

表 4.3 不同词嵌入的性能比较

嵌入模型	向量大小	Accuracy(%)	Precision(%)	Recall(%)	F1-Score(%)
CBOW	10	96.57	95.37	97.82	96.58
	20	97.83	98.34	97.31	97.82
	30	97.38	97.76	96.80	97.28
Skip-Gram	10	97.30	97.77	96.79	97.27
	20	98.40	98.52	98.33	98.43
	30	97.37	97.75	97.02	97.39

（5）与其他先进方法比较

为了全面地评估本章所提模型性能，本节将其与现有的其他先进的恶意代码

检测模型进行了详尽的性能对比。在本节中的实验中，采用了相同的数据集进行对比实验，通过对比各项关键指标，以评价模型的表现，如表 4.4 所示。

Catak 等人^[68]基于深度学习方法构建了恶意代码检测模型，通过嵌入层和两层 LSTM 对收集的 API 调用序列进行检测。Ding 等人^[73]提出了变长序列处理机制来表征过程中行为的 API 序列，并应用 Bert 将每个进程的 API 序列编码到节点嵌入中，最后，基于有向图和节点嵌入，使用图卷积网络挖掘深度语义信息。Zhang 等人^[74]从 API 名称和运行时参数中提取特征，使用哈希编码对 API 名称、类别和参数进行编码，并根据特征设置不同的维度大小。然后，将这些特征进一步连接并输入到 CNN 模型中，并应用 Bi-LSTM 来捕获 API 之间的连接。Chen 等人^[17]提出了一种恶意代码检测方法 CruParamer，根据经验规则和聚类技术评估参数对恶意行为的敏感性，并进一步以不同程度的敏感性来标记 API。然后，通过将 API 调用和标记的敏感标签连接起来，对语义信息对 API 进行编码。最后，将嵌入 API 序列输入深度神经网络，训练二元分类器来检测恶意代码。

结果显示，相较于未采用参数特征的模型，融入参数信息的模型在各项评估指标上表现优异，这验证了参数特征对于提升模型性能的作用。此外，本研究的模型在准确率和召回率方面展现出优势，分别达到了 98.4%和 98.33%，这表明本研究在参数处理方面能够保留和利用更多的有效信息，进而提升模型的整体性能。

表 4.4 与其他先进方法比较

	Parameters	Accuracy(%)	Precision(%)	Recall(%)	F1-Score(%)
Catak et al. ^[68]	×	96.85	97.20	96.42	96.81
Ding et al. ^[73]	×	97.82	97.84	97.83	97.82
Zhang et al. ^[74]	✓	97.25	96.52	98.05	97.28
Chen et al. ^[17]	✓	98.16	98.66	97.66	98.16
Proposed Model	✓	98.40	98.52	98.33	98.43

(6) t-SNE 可视化

鉴于深度学习模型在处理高维输入数据时，其内部学习机制及性能表现不易直观体现，本节采用非线性降维技术 t-SNE 方法，将较高维度的数据转化为易于理解的二维表示。具体而言，为了理解所提模型如何区分恶意代码与良性软件样本，运用 t-SNE (t-Distributed Stochastic Neighbor Embedding) 将模型处理后的高维数据进行非线性降维，使用主成分分析 (Principal Component Analysis, PCA) 将更高的维数据降为二维数据。

本节分别将模型的原始输入特征和检测模型倒数第二层输出特征通过 t-SNE 进行降维和可视化处理。图 4.12 展示了未经模型处理的原始测试样本经 t-SNE

降维后的分布状况，图 4.13 显示了测试样本经过模型分类后在二维空间中的映射情况。对比原始数据的 t-SNE 可视化结果，观察到在经过所提模型二分类处理后，t-SNE 图中恶意代码与良性代码样本在二维空间中形成的聚类边界更为明晰，且各自形成了紧凑且分离良好的簇群结构。这验证了本研究的模型在恶意代码检测的有效性。

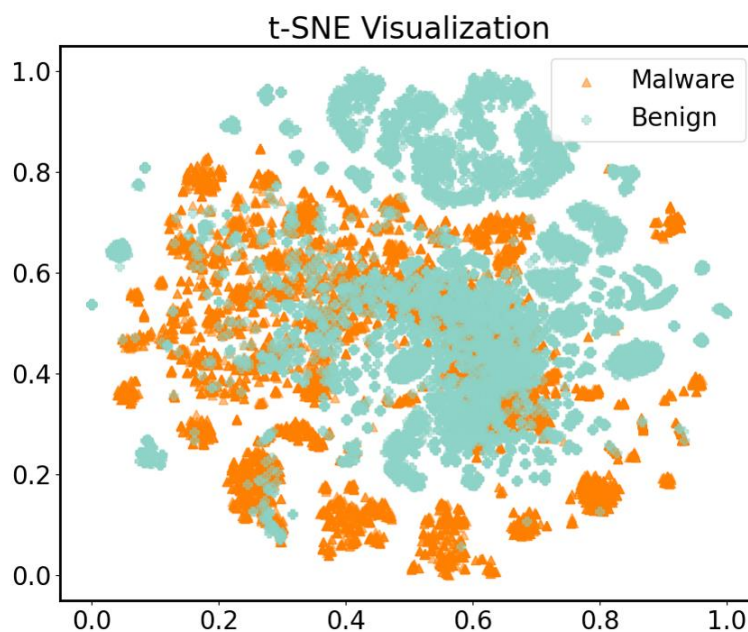


图 4.12 原始数据 t-SNE 图

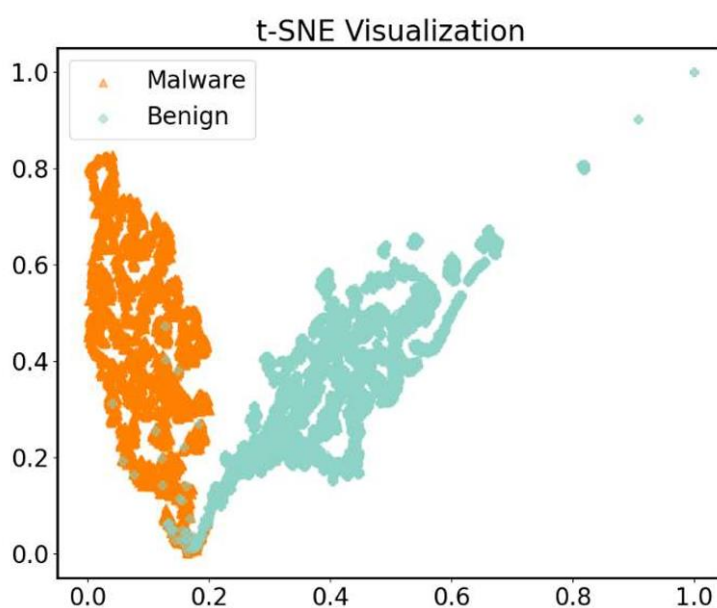


图 4.13 所提模型二分类后的 t-SNE 图

4.8 本章小结

本章提出了一种基于 API 异构参数增强的恶意代码检测的新方法，通过融合了程序 API 运行时参数的语义特征达到增强模型检测性能的效果。该方法使用段落向量化技术，将不同长度的 API 调用序列转化为统一维度的矢量形式，并设计使用双向时域卷积网络学习序列语义信息。同时，通过分析 API 参数的功能结构对参数进行划分，并利用轻量化模型学习参数之间的关联信息。最后，融合序列和参数特征对样本进行检测。试验结果表明，本章方法在相同数据集上的表现优于现有方法，具有出色的检测能力。

第5章 总结与展望

本章对本文开展的基于 API 行为特征的恶意代码检测工作进行总结，并对未来工作进行展望。

5.1 总结

随着信息技术的飞速发展与深度渗透，网络空间已成为现代社会运转不可或缺的一部分。恶意代码凭借其传染性、潜伏性、破坏性等特点，对个人隐私、企业资产乃至国家安全构成重大威胁。因此，对恶意代码进行快速且准确的检测，成为保障网络安全、维护社会稳定的关键一环。本文针对当前恶意代码检测研究所存在的问题，聚焦于动态分析方法，针对 API 行为特征展开研究。本文的主要贡献包括以下两个方面：

（1）传统恶意代码检测方法主要侧重于挖掘固定窗口内 API 调用的行为模式及频次特征，忽略了对 API 整体结构以及单个 API 调用内在特性的分析。针对现有工作对 API 调用序列信息分析不充分的问题，本文提出了一种基于 API 调用序列的恶意代码检测方法。首先，提取所需 API 调用序列，并通过预处理去除冗余调用，以精简数据保留有效特征，降低数据处理的复杂性。其次，在学习 API 调用序列语义信息的同时，还分析 API 调用名称结构的语义信息，并结合其统计特征，有效地解决了单一分析手段在信息捕捉上的局限性，提高检测的准确性。最后，将多尺度卷积结构与 BiGRU 相结合，实现从局部到全局的依赖关系的挖掘，提高了模型的检测准确率。

（2）现有恶意代码动态检测方法往往仅依赖于对 API 调用名称进行分析，而缺乏对传递给 API 的参数信息的研究，并且模型的计算复杂度较高。为了解决上述问题，本文提出了一种基于 API 异构参数增强的恶意代码检测方法。首先，删除冗余调用，并提取保留调用的参数信息。其次，使用段落向量化技术，将不同长度的 API 调用序列转化为统一维度的矢量形式，以解决因序列长度差异带来的特征向量稀疏性问题。进一步设计双向时域卷积网络，旨在以较低的计算复杂度有效处理长序列，同时消除梯度消失问题。然后，根据 API 参数的功能结构对其进行分解，并利用轻量化模型捕捉并理解参数间语义的内在联系，以有效降低计算资源消耗。最后，融合序列特征和参数特征，全面地理解程序行为，从而有效提高了模型的表示能力和泛化性能。

本文对提出的恶意代码检测方法进行了详细的实验设计和分析。实验数据表明,所提出的两种检测方法在恶意代码识别上具有较高的准确率和良好的稳定性。同时,还将上述方法与现有工作进行了对比评估,结果显示本文提出的方法在检测性能上均具有显著优势。

5.2 展望

本文提出了两种基于 API 行为特征的恶意代码检测方法,虽然获得了良好的性能,但仍然存在一些待解决的问题与局限性。为此,在未来的研究工作中需要着重于以下几个方面继续改进以实现进一步的优化和完善:

(1) 本文选取了针对 Windows 系统的恶意代码样本数据集,研究内容侧重于鉴别给定的样本是否为恶意。然而现实中的恶意样本会更加复杂,因此在未来的研究中,将对恶意代码的类型和家族进行检测,以更详尽地理解和应对恶意代码的多样性和变化。此外,考虑到现实网络环境中的恶意样本不仅限于 Windows 操作系统平台,未来的工作可以尝试将本文方法拓展至其他操作系统平台中进行测试,如 Android 平台,以验证其跨平台的适用性和有效性。

(2) 本文使用的是动态分析方法实现恶意样本的检测,以动态的 API 调用序作为主要研究对象。但是,一些高级的恶意代码可以检测到自己在虚拟环境中运行,从而采取逃避检测的行为,这导致无法捕获该类恶意代码的真实动态行为轨迹。为了解决上述问题,计划在后续研究中引入混合分析,通过将静态分析与动态分析技术相结合,以更全面地评估恶意代码的特性,从而实现了对恶意代码更为准确的检测。

(3) 新的恶意代码变体随着发现新的漏洞而出现,这会使传入的测试数据分布发生改变,与原始训练数据分布产生了偏离,最终导致分类器的性能随时间推移逐渐下降。为了解决上述问题,未来工作将对所提模型进行改进和优化,设计一个更具鲁棒性的特征处理方法,以更好地应对概念漂移问题,从而确保检测器在未来能够持续保持高效准确的恶意代码检测能力。

参考文献

- [1] Threat C. Sonicwall cyber threat report[R]. SonicWall, USA, Tech. Rep, 2023.
- [2] 汪嘉来,张超,戚旭衍,等.Windows 平台恶意软件智能检测综述[J].计算机研究与发展,2021,58(05):977-994.
- [3] Shalaginov A, Franke K. Automated intelligent multinomial classification of malware species using dynamic behavioural analysis[C]//2016 14th annual conference on privacy, security and trust (PST). IEEE, 2016: 70-77.
- [4] Xu L, Wang B, Wang L, et al. PLC-SEIFF: A programmable logic controller security incident forensics framework based on automatic construction of security constraints[J]. Computers & Security, 2020, 92: 101749.
- [5] Moskovitch R, Feher C, Tzachar N, et al. Unknown malcode detection using opcode representation[C]//European conference on intelligence and security informatics. Springer, Berlin, Heidelberg, 2008: 204-215.
- [6] Moskovitch R, Stopel D, Feher C, et al. Unknown malcode detection via text categorization and the imbalance problem[C]//2008 IEEE international conference on intelligence and security informatics. IEEE, 2008: 156-161.
- [7] Shabtai A, Moskovitch R, Elovici Y, et al. Detection of malicious code by applying machine learning classifiers on static features: A state-of-the-art survey[J]. information security technical report, 2009, 14(1): 16-29.
- [8] Sami A, Yadegari B, Rahimi H, et al. Malware detection based on mining API calls[C]//Proceedings of the 2010 ACM symposium on applied computing. 2010: 1020-1025.
- [9] 杨望,高明哲,蒋婷.一种基于多特征集成学习的恶意代码静态检测框架[J].计算机研究与发展,2021,58(05):1021-1034.
- [10] Chai Y, Du L, Qiu J, et al. Dynamic prototype network based on sample adaptation for few-shot malware detection[J]. IEEE Transactions on Knowledge and Data Engineering, 2022, 35(5): 4754-4766.
- [11] Hao J, Luo S, Pan L. EII-MBS: Malware family classification via enhanced adversarial instruction behavior semantic learning[J]. Computers & Security, 2022, 122: 102905.
- [12] Li S, Jiang L, Zhang Q, et al. A malicious mining code detection method based on multi-features fusion[J]. IEEE Transactions on Network Science and

- Engineering, 2022, 10 (5): 2731 - 2739.
- [13] Christodorescu M, Jha S, Kruegel C. Mining specifications of malicious behavior[C]//Proceedings of the the 6th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering. 2007: 5-14.
- [14] Tobiyama S, Yamaguchi Y, Shimada H, et al. Malware detection with deep neural network using process behavior[C]//2016 IEEE 40th annual computer software and applications conference (COMPSAC). IEEE, 2016, 2: 577-582.
- [15] Rosenberg I, Shabtai A, Rokach L, et al. Generic black-box end-to-end attack against state of the art API call based malware classifiers[C]//International Symposium on Research in Attacks, Intrusions, and Defenses. Springer, Cham, 2018: 490-510.
- [16] Ndibanje B, Kim K H, Kang Y J, et al. Cross-method-based analysis and classification of malicious behavior by api calls extraction[J]. Applied Sciences, 2019, 9(2): 239.
- [17] Chen X, Hao Z, Li L, et al. CruParamer: Learning on Parameter-Augmented API Sequences for Malware Detection[J]. IEEE Transactions on Information Forensics and Security, 2022, 17: 788-803.
- [18] Dabas N, Sharma P. MalAnalyser: An effective and efficient Windows malware detection method based on API call sequences[J]. Expert Systems with Applications, 2023, 230: 120756.
- [19] Damodaran A, Troia F D, Visaggio C A, et al. A comparison of static, dynamic, and hybrid analysis for malware detection[J]. Journal of Computer Virology and Hacking Techniques, 2017, 13(1): 1-12.
- [20] Zhao J, Zhang S, Liu B, et al. Malware detection using machine learning based on the combination of dynamic and static features[C]//2018 27th International Conference on Computer Communication and Networks (ICCCN). IEEE, 2018: 1-6.
- [21] Han W, Xue J, Wang Y, et al. MalDAE: Detecting and explaining malware based on correlation and fusion of static and dynamic characteristics[J]. computers & security, 2019, 83: 208-233.
- [22] Jeon J, Jeong B, Baek S, et al. Hybrid malware detection based on bi-lstm and spp-net for smart iot[J]. IEEE Transactions on Industrial Informatics, 2021, 18(7): 4830-4837.
- [23] David O E, Netanyahu N S. Deepsign: Deep learning for automatic malware

- signature generation and classification[C]//2015 International Joint Conference on Neural Networks (IJCNN). IEEE, 2015: 1-8.
- [24] Kang J, Jang S, Li S, et al. Long short-term memory-based malware classification method for information security[J]. Computers & Electrical Engineering, 2019, 77: 366-375.
- [25] Amer E, Zelinka I. A dynamic Windows malware detection and prediction method based on contextual understanding of API call sequence[J]. Computers & Security, 2020, 92: 101760.
- [26] 王兴凤, 黄琨茗, 张文杰. 基于 API 序列和卷积神经网络的恶意代码检测[J]. 信息安全研究, 2020, 6(3): 212-219.
- [27] Liu Y, Wang Y. A robust malware detection system using deep learning on API calls[C]//2019 IEEE 3rd Information Technology, Networking, Electronic and Automation Control Conference (ITNEC). IEEE, 2019: 1456-1460.
- [28] Wang P, Tang Z, Wang J. A novel few-shot malware classification approach for unknown family recognition with multi-prototype modeling[J]. Computers & Security, 2021, 106: 102273.
- [29] Kakisim A G, Gulmez S, Sogukpinar I. Sequential opcode embedding-based malware detection method[J]. Computers & Electrical Engineering, 2022, 98: 107703.
- [30] Ding Y, Yuan X, Tang K, et al. A fast malware detection algorithm based on objective-oriented association mining[J]. Computers & security, 2013, 39: 315-324.
- [31] Miao Q, Liu J, Cao Y, et al. Malware detection using bilayer behavior abstraction and improved one-class support vector machines[J]. International Journal of Information Security, 2016, 15: 361-379.
- [32] Galal H S, Mahdy Y B, Atiea M A. Behavior-based features model for malware detection[J]. Journal of Computer Virology and Hacking Techniques, 2016, 12: 59-67.
- [33] Eagle C. The IDA pro book[M]. San Francisco, CA: No Starch Press, 2011.
- [34] Bhardwaj V, Kukreja V, Sharma C, et al. Reverse engineering-a method for analyzing malicious code behavior[C]//2021 international conference on advances in computing, communication, and control (ICAC3). IEEE, 2021: 1-5.
- [35] Aslan Ö, Samet R. Investigation of possibilities to detect malware using existing tools[C]//2017 IEEE/ACS 14th International Conference on Computer Systems and Applications (AICCSA). IEEE, 2017: 1277-1284.

- [36] Poudyal S, Gupta K D, Sen S. PEFile analysis: a static approach to ransomware analysis[J]. Int J Forens Comput Sci, 2019, 1(34-39): 88.
- [37] Sikorski M, Honig A. Practical malware analysis: the hands-on guide to dissecting malicious software[M]. San Francisco, CA: No Starch Press, 2012.
- [38] Jamalpur S, Navya Y S, Raja P, et al. Dynamic malware analysis using cuckoo sandbox[C]//2018 Second international conference on inventive communication and computational technologies (ICICCT). IEEE, 2018: 1056-1060.
- [39] Babaagba K O, Adesanya S O. A study on the effect of feature selection on malware analysis using machine learning[C]//Proceedings of the 2019 8th international conference on educational and information technology. 2019: 51-55.
- [40] Ravi C, Manoharan R. Malware detection using windows api sequence and machine learning[J]. International Journal of Computer Applications, 2012, 43(17): 12-16.
- [41] Amer E, Zelinka I, El-Sappagh S. A multi-perspective malware detection approach through behavioral fusion of api call sequence[J]. Computers & Security, 2021, 110: 102449.
- [42] Maniriho P, Mahmood A N, Chowdhury M J M. API-MalDetect: Automated malware detection framework for windows based on API calls and deep learning techniques[J]. Journal of Network and Computer Applications, 2023, 218: 103704.
- [43] Aboaoja F A, Zainal A, Ghaleb F A, et al. Malware detection issues, challenges, and future directions: A survey[J]. Applied Sciences, 2022, 12(17): 8482.
- [44] Rosli N A, Yassin W, Faizal M A, et al. Clustering analysis for malware behavior detection using registry data[J]. International Journal of Advanced Computer Science and Applications (IJACSA), 2019, 10: 12.
- [45] Choudhary S P, Vidyarthi M D. A simple method for detection of metamorphic malware using dynamic analysis and text mining[J]. Procedia Computer Science, 2015, 54: 265-270.
- [46] Liang Y, Wang Q, Xiong K, et al. Robust detection of malicious URLs with self-paced wide & deep learning[J]. IEEE Transactions on Dependable and Secure Computing, 2021, 19(2): 717-730.
- [47] Ye Y, Li T, Adjero D, et al. A survey on malware detection using data mining techniques[J]. ACM Computing Surveys (CSUR), 2017, 50(3): 1-40.
- [48] Yang W, Gao M, Chen L, et al. RecMaL: Rectify the malware family label via hybrid analysis[J]. Computers & Security, 2023, 128: 103177.

- [49] 周飞燕, 金林鹏, 董军. 卷积神经网络研究综述[J]. 计算机学报, 2017, 40(6):1229-1251.
- [50] Elman J L. Finding structure in time[J]. Cognitive Science, 1990, 14(2): 179-211.
- [51] Tang J, Deng C, Huang G B. Extreme learning machine for multilayer perceptron[J]. IEEE transactions on neural networks and learning systems, 2015, 27(4): 809-821.
- [52] Staudemeyer R C, Morris E R. Understanding LSTM--a tutorial into long short-term memory recurrent neural networks[J]. arXiv preprint arXiv:1909.09586, 2019.
- [53] Dey R, Salem F M. Gate-variants of gated recurrent unit (GRU) neural networks[C]//2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS). IEEE, 2017: 1597-1600.
- [54] Bai S, Kolter J Z, Koltun V. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling[J]. arXiv preprint arXiv:1803.01271, 2018.
- [55] Türkoğlu A S, Erkmén B, Eren Y, et al. Integrated Approaches in Resilient Hierarchical Load Forecasting via TCN and Optimal Valley Filling Based Demand Response Application[J]. Applied Energy, 2024, 360: 122722.
- [56] 杨宇,谷宇恒.网络安全态势感知综述[J].科学技术与工程,2022,22(34):15011-15019.
- [57] Burnap P, French R, Turner F, et al. Malware classification using self organising feature maps and machine activity data[J]. computers & security, 2018, 73: 399-410.
- [58] Ucci D, Aniello L, Baldoni R. Survey of machine learning techniques for malware analysis[J]. Computers & Security, 2019, 81: 123-147.
- [59] Wang S, Zhou W, Jiang C. A survey of word embeddings based on deep learning[J]. Computing, 2020, 102(3): 717-740.
- [60] Alami N, Meknassi M, En-Nahnahi N. Enhancing unsupervised neural networks based text summarization with word embedding and ensemble learning[J]. Expert systems with applications, 2019, 123: 195-211.
- [61] Rezaeinia S M, Rahmani R, Ghodsi A, et al. Sentiment analysis based on improved pre-trained word embeddings[J]. Expert Systems with Applications, 2019, 117: 139-147.
- [62] Martinčić-Ipšić S, Miličić T, Todorovski L. The influence of feature representation of text on the performance of document classification[J]. Applied

- Sciences, 2019, 9(4): 743.
- [63] Shelton C P, Koopman P, DeVale K. Robustness testing of the Microsoft Win32 API[C]//Proceeding International Conference on Dependable Systems and Networks. DSN 2000. IEEE, 2000: 261-270.
- [64] Li C, Lv Q, Li N, et al. A novel deep framework for dynamic malware detection based on API sequence intrinsic features[J]. Computers & Security, 2022, 116: 102686.
- [65] kericwy1337. Datacon2019 Malicious Code DataSet Stage1[EB/OL]. (2019-05-10)[2023-05-01]. <https://github.com/kericwy1337/Datacon2019-Malicious-Code-DataSet-Stage1>
- [66] Fawcett T. An introduction to ROC analysis[J]. Pattern recognition letters, 2006, 27(8): 861-874.
- [67] Kolosnjaji B, Zarras A, Webster G, et al. Deep learning for classification of malware system call sequences[C]//AI 2016: Advances in Artificial Intelligence: 29th Australasian Joint Conference, Hobart, TAS, Australia, December 5-8, 2016, Proceedings 29. Springer International Publishing, 2016: 137-149.
- [68] Catak F O, Yazı A F, Elezaj O, et al. Deep learning based Sequential model for malware analysis using Windows exe API Calls[J]. PeerJ Computer Science, 2020, 6: e285.
- [69] Le Q, Mikolov T. Distributed representations of sentences and documents[C]//International conference on machine learning. PMLR, 2014: 1188-1196.
- [70] Aman H, Amasaki S, Yokogawa T, et al. A doc2vec-based assessment of comments and its application to change-prone method analysis[C]//2018 25th Asia-Pacific Software Engineering Conference (APSEC). IEEE, 2018: 643-647.
- [71] Strubell E, Ganesh A, McCallum A. Energy and policy considerations for deep learning in NLP[J]. arXiv preprint arXiv:1906.02243, 2019.
- [72] Tan M, Le Q. Efficientnet: Rethinking model scaling for convolutional neural networks[C]//International conference on machine learning. PMLR, 2019: 6105-6114.
- [73] Ding Z, Xu H, Guo Y, et al. Mal-Bert-GCN: Malware Detection by Combining Bert and GCN[C]//2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom). IEEE, 2022: 175-183.

- [74] Zhang Z, Qi P, Wang W. Dynamic malware analysis with feature engineering and feature learning[C]//Proceedings of the AAAI conference on artificial intelligence. 2020, 34(01): 1210-1217.